

1 Algorithm

1.1 LIS

```
1 // Longest increasing subsequence
2 int LIS(const vector<int>& s) {
3     // use binary search to update the smallest
4     // element
5     // that ends in a subsequence of length d
6     vector<int> v{};
7     v.push_back(s[0]);
8     for (int i{1}; i < s.size(); ++i)
9         if (s[i] > v.back()) v.push_back(s[i]);
10        else *lower_bound(v.begin(), v.end(),
11                           s[i]) = s[i];
12    return v.size();
13 }
```

1.2 LCS

```
1 // Longest common subsequence
2 int LCS(const vector<int>& a, const vector<
3         int>& b) {
4     vector<vector<int>> dp(a.size() + 1,
5                             vector<int>(b.size() + 1));
6     for (int i{1}; i <= int(a.size()); ++i)
7         for (int j{1}; j <= int(b.size()); ++j) {
8             dp[i][j] = max(dp[i][j], max(dp[i - 1][j], dp[i][j - 1]));
9             if (a[i - 1] == b[j - 1]) dp[i][j] = max(dp[i][j], dp[i - 1][j - 1] + 1);
10        }
11    return dp[a.size()][b.size()];
12 }
```

2 Basic

2.1 mod helper function

```
1 int add(int i, int j) {
2     if ((i += j) >= MOD)
3         i -= MOD;
4     return i;
5 }
6
7 int sub(int i, int j) {
8     if ((i -= j) < 0)
9         i += MOD;
10    return i;
11 }
```

2.2 self-defined-pq-operator

```
1 auto cmp = [](int a, int b) {
2     return a > b;
3 };
4 priority_queue<int, vector<int>, decltype(
5     cmp)> pq(cmp);
```

2.3 generating all subsets

```
1 for (int b = 0; b < (1<<n); b++) {
2     vector<int> subset;
3     for (int i = 0; i < n; i++) {
4         if (b & (1<<i)) subset.push_back(vc[i]);
5     }
6 }
```

2.4 tuple

```
1 int main() {
2     tuple<int, double, string> myTuple(100,
3     3.14, "hello world");
4     tuple<int, double, string> anotherTuple
5     = make_tuple(100, 18, "Tom");
6     cout << get<0>(myTuple) << "\n"; // 取得
7     // 第一個元素
8     auto [a, b, c] = myTuple;
9     get<1>(myTuple) = 5.43; // 修改第二個元
10    // 素的值
11
12     int a;
13     double b;
14     string c;
15     tie(a, b, c) = myTuple;
16 }
```

2.5 memset

```
1 memset(a, 0, sizeof(a)); // 0
2 memset(a, 0x3f3f3f3f, sizeof(a)); // INF
```

2.6 submask enumeration

```
1 // O(3^n)
2
3 int m = 0b10110; // binary: 10110, decimal:
4 // 22
5 cout << "Mask: " << bitset<5>(m) << " (" <<
6     m << ")\\n";
```

```
7 // Enumerate all submasks
8 for (int s = m; s; s = (s - 1) & m) {
9     cout << bitset<5>(s) << " (" << s << ")
10    \\n";
11 }
12 // Optionally include the empty submask
13 cout << bitset<5>(0) << " (0)\\n";
```

2.7 stringstream split by comma

```
1 while (std::getline(ss, segment, ',')) {
2     segments.push_back(segment);
3 }
```

3 DP

3.1 walk

```
1 /*
2 DP on graphs - O(N^3 Log K)
3 給定一個簡單的有向圖 G，具有 N 個頂點，編號
4 為 1, 2, ..., N。
5 對於任意 i, j (1 ≤ i, j ≤ N)，給定整數 a_{i,j}，表示
6 是否存在從頂點 i 指向頂點 j 的有向邊。若 a_{i,j} = 1，則存在邊；若 a_{i,j} = 0，則不存在。
7 求圖中長度為 K 的不同有向路徑數目，對 10^9+7
8 取模。路徑可重複通過相同邊（即允許重複
9 邊）。
10 注意：當我們將鄰接矩陣 m 與 m 相乘時，得到的是
11 長度為 2 的路徑數；若取 m 的 p 次方 m^p，則其 (i, j)
12 元素表示從 i 到 j 的長度為 p 的路徑數。
13
14 */
15 void solve() {
16     int n, k;
17     cin >> n >> k;
18     vector<vector<int>> m(n, vector<int>(n));
19
20     for (int i = 0; i < n; ++i) {
21         for (int j = 0; j < n; ++j) {
22             cin >> m[i][j];
23         }
24     }
25
26     Matrix<int> mat(m);
27     mat = power(mat, k);
28     int ans = 0;
29     for (int i = 0; i < n; ++i) {
30         for (int j = 0; j < n; ++j) {
31             ans += mat[i][j];
32         }
33     }
34 }
```

```
28 ans += mat.mat[i][j];
29 ans %= MOD;
30 }
31 }
32 cout << ans << "\\n";
33 }
```

3.2 grouping

```
1 /*
2 狀態 DP - O(3^N * 2^N * N^2)
3 有 N 隻兔子，編號為 1, 2, ..., N。
4
5 對於每一對 i, j (1 ≤ i, j ≤ N)，兔子的相容
6 度由整數 a_{i,j} 描述。這裡 a_{i,i} = 0 對於
7 每個 i (1 ≤ i ≤ N)，且 a_{i,j} = a_{j,i} 對於任
8 意 i 與 j (1 ≤ i, j ≤ N)。
9
10 A 將 N 隻兔子分成若干個群組。每隻兔子必須且
11 僅屬於一個群組。分群後，對於每一對 i 與
12 j (1 ≤ i < j ≤ N)，若兔子 i 與 j 屬於同一群
13 組，A 即可獲得 a_{i,j} 分。
14
15 求 A 能獲得的最大總分。
16
17 令 cost[S] 表示將集合 S 中的所有兔子放在同一
18 群組時所得到的分數。此值可在 O(2^N * N^2)
19 時間內計算。
20
21 接著我們計算 dp[S]，表示對集合 S 中的兔子進
22 行分群時所能得到的最大分數。
23
24 */
25 void solve() {
26     int n;
27     cin >> n;
28     vector<vector<int>> a(n, vector<int>(n));
29
30     vector<int> cost(1<<n, 0);
31     vector<int> dp(1<<n, 0);
32
33     for (int i = 0; i < n; ++i) {
34         for (int j = 0; j < n; ++j) {
35             cin >> a[i][j];
36         }
37     }
38
39     // backtrack all subset
40     for (int b = 0; b < (1<<n); ++b) {
41         vector<int> subset;
42         for (int i = 0; i < n; ++i) {
43             if (b & (1<<i)) {
44                 for (const int& j : subset) {
45                     cost[b] += a[i][j];
46                 }
47                 subset.pb(i);
48             }
49         }
50     }
```

```

41 // dp
42 for (int i = 0; i < (1<<n); ++i) {
43     int j = ((1<<n) - 1) ^ i;
44     for (int s = j; s != 0; s = (s - 1)
45         & j) {
46         dp[i ^ s] = max(dp[i ^ s], dp[i]
47             + cost[s]);
48     }
49 }
50 cout << dp[(1<<n) - 1] << "\n";

```

3.3 matching

```

1 /*
2 Bitmask DP - O(N * 2^N)
3 有 N 個男人和 N 個女人，分別編號為 1, 2, ...,
4   N。
5 對於每個 i, j (1 ≤ i, j ≤ N)，男人 i 和女人
6   j 的相容性由整數 a[i][j] 給出。
7 如果 a[i][j] = 1，則男人 i 和女人 j 是相容
8   的；
9 如果 a[i][j] = 0，則不是。
10 A 正在嘗試組成 N 對，每對由一個相容的男人和
11   女人組成。在這裡，每個男人和每個女人必須
12   恰好屬於一對。
13 求 A 可以組成 N 對的方法數，結果對 10^9 + 7
14   取模。
15 定義 dp[S] 為將集合 S 中的女性與前 |S| 個男
16   性配對的方法數。
17 */
18 const int maxn = 21;
19 const int MOD = 1e9 + 7;
20 int n;
21 int grid[maxn][maxn];
22 void solve() {
23     cin >> n;
24     memset(dp, 0, sizeof(dp));
25
26     for (int i = 0; i < n; ++i) {
27         for (int j = 0; j < n; ++j) {
28             cin >> grid[i][j];
29         }
30     }
31
32     dp[0] = 1;
33     for (int s = 0; s < (1 << n); ++s) {
34         int ps = __builtin_popcount(s);
35         for (int w = 0; w < n; ++w) {
36             if ((s & (1 << w)) || !grid[ps][
37                 w]) {
38                 continue;

```

```

39         dp[s | (1 << w)] += dp[s];
40         dp[s | (1 << w)] %= MOD;
41     }
42 }
43 cout << dp[(1 << n) - 1] << "\n";
44 }
45
3.4 elevator rides
1 /*
2 狀態 DP - O(2^N)
3 有 n 個人想要搭電梯到樓頂，建築物只有一部電
4   梯。你知道每個人的體重以及電梯的最大允許
5   載重。最少需要搭乘多少次電梯？
6 定義 dp[S] = {r, w}，其中 r 是將集合 S 中的
7   所有人送到樓頂所需的最少電梯次數，w 是最
8   後一次電梯所載人的總重量。
9 */
10 void solve() {
11     int n, x;
12     cin >> n >> x;
13     vector<int> w(n);
14     vector<pii> dp(1<<n, {INF, INF});
15
16     for (int i = 0; i < n; ++i) {
17         cin >> w[i];
18     }
19
20     dp[0] = {1, 0};
21     for (int b = 1; b < (1<<n); ++b) {
22         for (int i = 0; i < n; ++i) {
23             if (b & (1<<i)) {
24                 auto [r_prev, w_prev] = dp[b
25                     ^ (1<<i)];
26                 pii can;
27                 if (w_prev + w[i] <= x) {
28                     can = {r_prev, w_prev +
29                         w[i]};
30                 } else {
31                     can = {r_prev + 1, w[i
32                         ]};
33                 }
34                 dp[b] = min(dp[b], can);
35             }
36         }
37     }
38     cout << dp[(1<<n) - 1].first << "\n";
39 }

```

3.5 digit sum

```

1 /*
2 Digit DP - O(|K| * D)

```

```

3 計算在 1 到 K (含) 之間，滿足其十進位數字和
4   為 D 的倍數的整數數量，答案對 10^9+7 取
5   模。
6 令 dp[i][j] 表示在已確定前 i 位數字的情況
7   下，構成長度為 |K| 的數字且目前數字和
8   mod D 等於 j 的方法數。
9 */
10 const int MOD = 1e9 + 7;
11 int dp[10001][101][2];
12
13 void solve() {
14     string K;
15     int D;
16     cin >> K >> D;
17     int len = K.size();
18     memset(dp, 0, sizeof(dp));
19     dp[0][0][1] = 1;
20
21     for (int i = 1; i <= len; ++i) {
22         int limit = K[i - 1] - '0';
23         for (int s = 0; s < D; ++s) {
24             for (int flag = 0; flag <= 1; ++
25                 flag) {
26                 int ways = dp[i - 1][s][flag
27                     ];
28                 if (ways == 0) continue;
29                 int max_d = (flag ? limit :
30                     9);
31                 for (int d = 0; d <= max_d;
32                     ++d) {
33                     int rs = (s + d) % D;
34                     int rflag = (flag && d
35                         == max_d ? 1 : 0);
36                     dp[i][rs][rflag] += ways
37                         ;
38                     dp[i][rs][rflag] %= MOD;
39                 }
40             }
41         }
42     }
43
44     int ans = (dp[len][0][0] + dp[len
45         ][0][1]) % MOD;
46     ans = (ans - 1 + MOD) % MOD;
47     cout << ans << "\n";
48 }

```

3.6 sushi

```

1 /*
2 期望值 DP - O(N^3)
3 有 N 盤壽司，編號從 1 到 N。第 i 盤最初有 a_i
4   (1 ≤ a_i ≤ 3) 塊壽司。
5 太郎不斷擲一顆編號 1 到 N 的骰子。如果結果是
6   第 i 盤且該盤還有壽司，他就吃掉一塊；否
7   則什麼也不做。

```

```

8 dp[x][y][z] 代表還有
9 x 盤剩 1 塊壽司、
10 y 盤剩 2 塊壽司、
11 z 盤剩 3 塊壽司時的期望擲骰次數。
12 */
13 const int maxn = 301;
14 double dp[maxn][maxn][maxn];
15 int n;
16
17 double dfs(int x, int y, int z) {
18     if (x < 0 || y < 0 || z < 0) return 0;
19     if (x == 0 && y == 0 && z == 0) return
20         0;
21     if (dp[x][y][z] > 0) return dp[x][y][z];
22     double ans = n + x * dfs(x - 1, y, z)
23         + y * dfs(x + 1, y - 1, z)
24         + z * dfs(x, y + 1, z -
25             1);
26     return dp[x][y][z] = ans / (x + y + z);
27 }
28
29 void solve() {
30     cin >> n;
31     vector<int> a(n);
32     memset(dp, -1, sizeof(dp));
33     vector<int> freq(4, 0);
34
35     for (int i = 0; i < n; ++i) {
36         cin >> a[i];
37         freq[a[i]]++;
38     }
39
40     cout << fixed << setprecision(10) << dfs
41         (freq[1], freq[2], freq[3]) << "\n";
42 }

```

3.7 permutation

```

1 /*
2 抽象 DP - O(N^2)
3 設 N 為正整數。給定一個長度為 N-1 的字串 s，
4   字元僅包含 '<' 與 '>'。
5 求滿足條件的排列 (p1, p2, ..., pN) (即 1 到 N
6   的排列) 數量，答案對 10^9+7 取模：
7 對於每個 i (1 ≤ i ≤ N-1)，若 s 的第 i 個字
8   元為 '<'，則要求 p_i < p_{i+1}；若為 '>'，
9   則要求 p_i > p_{i+1}。
10 令 dp[i][j] 表示：在考慮前 i 個比較符號 (即
11   構成長度為 i+1 的排列) 且最後一個元素為
12   j 的有效排列數量。
13 */
14 void solve() {
15     int n;

```

```

14 string s;
15 cin >> n >> s;
16 vector<vector<int>> dp(n + 1, vector<int>
17     >(n + 1, 0));
18 vector<int> prefix(n + 1, 0);
19 dp[1][0] = 1;
20
21 for (int i = 2; i <= n; ++i) {
22     for (int k = 0; k < n; ++k) {
23         prefix[k + 1] = prefix[k] + dp[i
24             - 1][k];
25     }
26     for (int j = 0; j < i; ++j) {
27         if (s[i - 2] == '>') {
28             dp[i][j] += prefix[i - 1] -
29                 prefix[j];
30             dp[i][j] %= MOD;
31             /*
32             for (int k = j; k < i - 1;
33                 ++k) {
34                 dp[i][j] += dp[i - 1][k
35                     ];
36             }
37             */
38         }
39         else {
40             dp[i][j] += prefix[j];
41             dp[i][j] %= MOD;
42             /*
43             for (int k = 0; k < j; ++k) {
44                 dp[i][j] += dp[i - 1][k
45                     ];
46             }
47             */
48         }
49     }
50 }
51 int ans = 0;
52 for (int j = 0; j < n; ++j) {
53     ans += dp[n][j];
54     ans %= MOD;
55 }
56 cout << ans << "\n";
57 }

```

3.8 independent set

```

1 /*
2 DP on Trees - O(N)
3 有一棵含 N 個頂點的樹，頂點編號為 1, 2, ..., N。
4 對於每個 i (1 ≤ i ≤ N-1)，第 i 條邊連接
5 頂點 x_i 和 y_i。
6
7 A 決定將每個頂點塗成白色或黑色，但不允許兩個
8 相鄰的頂點同時為黑色。
9
10 求將頂點塗色的方案數，對 10^9+7 取模。
11
12 設 dp[i][j] 表示以節點 i 為根的子樹中，在節
13 點 i 顏色為 j 時的塗色方案數（例如 j=0
14 表示白色，j=1 表示黑色）。

```

```

10 /*
11
12 const int maxn = 100001;
13 vector<int> adj[maxn];
14 int f[maxn][2];
15 const int MOD = 1e9 + 7;
16
17 void dp(int u, int p) {
18     for (int v : adj[u]) {
19         if (v != p) {
20             dp(v, u);
21             f[u][0] = (f[v][0] + f[v][1]) %
22                 MOD;
23             f[u][1] = f[v][0] * f[u][1] %
24                 MOD;
25         }
26     }
27 }
28
29 void solve() {
30     int n;
31     cin >> n;
32
33     for (int i = 0; i < n; ++i) {
34         f[i][0] = f[i][1] = 1;
35     }
36
37     for (int i = 0; i < n - 1; ++i) {
38         int u, v;
39         cin >> u >> v;
40         u--, v--;
41         adj[u].pb(v);
42         adj[v].pb(u);
43     }
44
45     dp(0, -1);
46
47     cout << (f[0][0] + f[0][1]) % MOD << "\n";
48 }

```

3.9 counting tower

```

1 /*
2 狀態機 DP - O(N)
3 你的任務是建造一座寬度為 2、高度為 n 的塔。
4 你有無限數量寬度與高度為整數的方塊。
5
6 dp[i][0] = 高度為 i 的塔中，頂層為一個寬度為
7 2 的方塊（即該層由跨越兩欄的單一方塊覆
8 蓋）的塔的數量。
9
10 dp[i][1] = 高度為 i 的塔中，頂層在該層有兩個
11 寬度為 1 的方塊（每欄各一個）的塔的數
12 量。
13
14 */
15
16 void solve() {
17     long long n;
18     cin >> n;
19     dp[1][0] = 1;
20     dp[1][1] = 1;

```

```

14 for (int i = 2; i <= n; ++i) {
15     dp[i][0] = (4 * dp[i-1][0] + dp[i
16         - 1][1]) % mod;
17     dp[i][1] = (dp[i-1][0] + 2 * dp[i
18         - 1][1]) % mod;
19 }
20 cout << (dp[n][0] + dp[n][1]) % mod <<
21 endl;
22 }

```

3.10 counting numbers

```

1 /*
2 區間 DP - O(digits*10)
3 您的任務是計算在區間 a 到 b 之間，沒有任何相
4 鄰兩位數字相同的整數的個數。
5
6 定義 dp[pos][prev_digit][is_tight][
7 is_started] 為從位置 pos 到結尾的有效整
8 數數量。
9
10 其中 prev_digit 是位置 pos-1 的數字。
11 is_tight 表示是否受原數字前綴的限制。
12 is_started 表示是否已開始構成有效數字（用以
13 避免計入前導零）。
14
15 */
16
17 // dp[pos][prev_digit][is_tight][is_started]
18 int dp[20][10][2][2];
19 string s;
20
21 int f(int pos, int prev_digit, bool is_tight
22     , bool is_started) {
23     if (pos == (int)s.size()) {
24         return 1;
25     }
26     if (dp[pos][prev_digit][is_tight][
27         is_started] != -1) {
28         return dp[pos][prev_digit][is_tight
29             ][is_started];
30     }
31
32     int ans = 0;
33     int max_d = is_tight ? (s[pos] - '0') :
34         9;
35     for (int d = 0; d <= max_d; ++d) {
36         if (is_started && d == prev_digit) {
37             continue;
38         }
39
40         bool new_is_started = is_started ||
41             (d > 0);
42         bool new_is_tight = is_tight && (d
43             == max_d);
44         ans += f(pos + 1, d, new_is_tight,
45             new_is_started);
46     }
47
48     return dp[pos][prev_digit][is_tight][
49         is_started] = ans;
50 }
51 }

```

```

38 int count(int x) {
39     if (x < 0) return 0;
40     s = to_string(x);
41     memset(dp, -1, sizeof(dp));
42     return f(0, 0, true, false);
43 }
44
45 void solve() {
46     int a, b;
47     cin >> a >> b;
48
49     int ca = count(a - 1);
50     int cb = count(b);
51     cout << cb - ca << "\n";
52 }

```

4 Data Structure

4.1 undo disjoint set

```

1 struct DisjointSet {
2     // save() is like recursive
3     // undo() is like return
4     int n, fa[MXN], sz[MXN];
5     vector<pair<int*, int*>> h;
6     vector<int> sp;
7     void init(int tn) {
8         n=tn;
9         for (int i=0; i<n; i++) sz[fa[i]=i]=1;
10        sp.clear(); h.clear();
11    }
12    void assign(int *k, int v) {
13        h.PB({k, *k});
14        *k=v;
15    }
16    void save() { sp.PB(SZ(h)); }
17    void undo() {
18        assert(!sp.empty());
19        int last=sp.back(); sp.pop_back();
20        while (SZ(h)!=last) {
21            auto x=h.back(); h.pop_back();
22            *x.F=x.S;
23        }
24    }
25    int f(int x) {
26        while (fa[x]!=x) x=fa[x];
27        return x;
28    }
29    void uni(int x, int y) {
30        x=f(x); y=f(y);
31        if (x==y) return;
32        if (sz[x]<sz[y]) swap(x, y);
33        assign(&sz[x], sz[x]+sz[y]);
34        assign(&fa[y], x);
35    }
36 } djs;

```

4.2 lazy propagation

```

1 // Lazy Propagation Segment Tree supporting
  range add, range set, and range sum
  queries
2
3 int n;
4 int sz;
5 struct Node {
6     int sum = 0;
7     int lazy_add = 0;
8     int lazy_set = 0;
9     bool has_set = false;
10 };
11 vector<Node> tree;
12
13 void build(const vector<int>& a) {
14     tree.resize(sz << 1);
15     for (int i = 0; i < sz; ++i) {
16         tree[i + sz].sum = i < n ? a[i] : 0;
17     }
18     for (int i = sz - 1; i >= 1; --i) {
19         tree[i].sum = tree[i << 1].sum +
20             tree[i << 1 | 1].sum;
21     }
22
23 void apply_set(int idx, int l, int r, int
24     val) {
25     tree[idx].sum = (r - l + 1) * val;
26     tree[idx].lazy_set = val;
27     tree[idx].lazy_add = 0;
28     tree[idx].has_set = true;
29 }
30
31 void apply_add(int idx, int l, int r, int
32     val) {
33     if (tree[idx].has_set) {
34         tree[idx].lazy_set += val;
35     }
36     else {
37         tree[idx].lazy_add += val;
38     }
39     tree[idx].sum += (r - l + 1) * val;
40 }
41
42 void push(int idx, int l, int r) {
43     if (l == r) return;
44
45     int mid = l + (r - l) / 2;
46     int left = idx << 1, right = idx << 1 |
47         1;
48
49     if (tree[idx].has_set) {
50         // propagate the set operation
51         apply_set(left, l, mid, tree[idx].
52             lazy_set);
53         apply_set(right, mid + 1, r, tree[
54             idx].lazy_set);
55         tree[idx].has_set = false;
56         tree[idx].lazy_set = 0;
57         tree[idx].lazy_add = 0;
58     }
59     else if (tree[idx].lazy_add != 0) {
60         // propagate the add operation

```

```

56     apply_add(left, l, mid, tree[idx].
57         lazy_add);
58     apply_add(right, mid + 1, r, tree[
59         idx].lazy_add);
60     tree[idx].lazy_add = 0;
61 }
62
63 void range_set(int idx, int l, int r, int ql
64     , int qr, int val) {
65     if (qr < l || ql > r) return;
66     if (ql <= l && r <= qr) {
67         apply_set(idx, l, r, val);
68         return;
69     }
70     push(idx, l, r);
71     int mid = (l + r) / 2;
72     range_set(idx << 1, l, mid, ql, qr, val);
73     range_set(idx << 1 | 1, mid + 1, r, ql,
74         qr, val);
75     tree[idx].sum = tree[idx << 1].sum +
76         tree[idx << 1 | 1].sum;
77 }
78
79 void range_add(int idx, int l, int r, int ql
80     , int qr, int val) {
81     if (ql > r || qr < l) return;
82     if (ql <= l && r <= qr) {
83         apply_add(idx, l, r, val);
84         return;
85     }
86     push(idx, l, r);
87     int mid = (l + r) / 2;
88     range_add(idx << 1, l, mid, ql, qr, val);
89     range_add(idx << 1 | 1, mid + 1, r, ql,
90         qr, val);
91     tree[idx].sum = tree[idx << 1].sum +
92         tree[idx << 1 | 1].sum;
93 }
94
95 int range_sum(int idx, int l, int r, int ql,
96     int qr) {
97     if (qr < l || ql > r) return 0;
98     if (ql <= l && r <= qr) return tree[idx
99         ].sum;
100     push(idx, l, r);
101     int mid = (l + r) / 2;
102     return range_sum(idx << 1, l, mid, ql,
103         qr) +
104         range_sum(idx << 1 | 1, mid + 1,
105             r, ql, qr);
106 }
107
108 void solve() {
109     int q;
110     cin >> n >> q;
111     vector<int> a(n);
112
113     sz = 1;
114     while (sz < n) sz <= 1;
115     for (int i = 0; i < n; ++i) {
116         cin >> a[i];
117     }
118
119     build(a);
120
121     while (q--) {
122         int mode, l, r, x;
123         cin >> mode >> l >> r;
124         l--, r--;
125         if (mode == 1) {
126             cin >> x;
127             range_add(1, 0, sz - 1, l, r, x);
128         }
129         else if (mode == 2) {
130             cin >> x;
131             range_set(1, 0, sz - 1, l, r, x);
132         }
133         else {
134             cout << range_sum(1, 0, sz - 1,
135                 l, r) << "\n";
136         }
137     }
138 }

```

4.3 Lazy heap

```

1 template<typename T, typename Compare = less
2     <T>>
3 struct lp {
4     priority_queue<T, vector<T>, Compare> pq;
5     unordered_map<T, int> cnt;
6     size_t sz = 0;
7     void real_remove() {
8         while (!pq.empty() && cnt[pq.top()] >
9             0) {
10             cnt[pq.top()]--;
11             pq.pop();
12         }
13     }
14     size_t size() {
15         return sz;
16     }
17     void remove(T n) {
18         cnt[n]++;
19         sz--;
20     }
21     T top() {
22         real_remove();
23         return pq.top();
24     }
25     void push(T n) {
26         pq.push(n);
27         sz++;
28     }
29     void pop() {
30         real_remove();
31         sz--;
32         pq.pop();
33     }
34 };

```

4.4 Fenwick tree (BIT)

```

1 struct Fenwick {
2     int n;
3     vector<int> bit, prefix;
4     Fenwick(int _n) : n(_n), bit(n + 1, 0),
5         prefix(n + 1, 0) {}
6     void update(int i, int delta) {
7         for (; i <= n; i += i & -i) {
8             bit[i] += delta;
9         }
10    }
11    int query(int i) {
12        int ans = 0;
13        for (; i >= 1; i -= i & -i) {
14            ans += bit[i];
15        }
16        return ans;
17    }
18    int range(int l, int r) {
19        return query(r) - query(l - 1);
20    }
21 };
22
23 int main() {
24     int n = 5;
25     vector<int> a = {0, 1, 2, 3, 4, 5}; //
26     // 1-indexed
27     Fenwick ft(n);
28     for (int i = 1; i <= n; ++i) ft.update(i,
29         a[i]);
30
31     cout << ft.query(3) << "\n"; // 6
32     cout << ft.range(2, 4) << "\n"; // 9
33
34     ft.update(3, 10); //
35     // a[3] += 10
36     cout << ft.query(3) << "\n"; // 16
37     cout << ft.range(1, 5) << "\n"; // 25
38 }

```

4.5 BIT - range update + range prefix sum

```

1 int n, q;
2
3 void solve() {
4     cin >> n >> q;
5     Fenwick bit_values(n), bit_count(n);
6
7     while (q--) {
8         int mode, l, r, val;
9         cin >> mode;
10        if (mode == 0) {
11            cin >> l >> r >> val;
12            bit_values.update(l, val);
13            bit_count.update(l, val * (l -
14                1));
15            bit_values.update(r + 1, -val);
16            bit_count.update(r + 1, -val * r);
17        }
18    }
19 }

```

```

17     else {
18         cin >> l >> r;
19         int pref_l = bit_values.query(l
20             - 1) * (1 - 1) - bit_count.
21             query(l - 1);
22         int pref_r = bit_values.query(r)
23             * r - bit_count.query(r);
24         cout << pref_r - pref_l << "\n";
25     }
26 }

```

4.6 Order Statistics Tree

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 #include <ext/pb_ds/tree_policy.hpp>
3 using namespace __gnu_pbds;
4
5 template <class T>
6 using ordered_set =
7     tree<T, null_type, less<T>, rb_tree_tag,
8         tree_order_statistics_node_update>;
9
10 int main() {
11     int n;
12     cin >> n;
13     ordered_set<int> st;
14     vector<int> p(n);
15     for (int i = 0; i < n; i++) {
16         st.insert(i);
17         cin >> p[i];
18     }
19     for (int i = 0; i < n; i++) {
20         int ind;
21         cin >> ind;
22         ind--;
23         int pos = *st.find_by_order(ind);
24         cout << st.order_of_key(pos) << ' ';
25         st.erase(pos);
26         cout << p[pos] << (i == n - 1 ? '\n' : ' ');
27     }
28 }

```

4.7 Trie (Prefix tree)

```

1 struct trie {
2     int n = 0;
3     trie *a[2];
4     trie() {
5         a[0] = a[1] = nullptr;
6     }
7
8     void insert(int k) {
9         trie *curr = this;
10        for (int i = 63; i >= 0; --i) {
11            bool bit = (k & (1LL << i)) > 0;
12            if (curr->a[bit] == nullptr) {
13                curr->a[bit] = new trie();
14            }

```

```

15        curr = curr->a[bit];
16        curr->n++;
17    }
18 }
19
20 void erase(int k) {
21     trie *curr = this;
22     for (int i = 63; i >= 0; --i) {
23         bool bit = (k & (1LL << i)) > 0;
24         curr = curr->a[bit];
25         curr->n--;
26     }
27 }
28
29 int query(int k) {
30     trie *curr = this;
31     int x = 0;
32     for (int i = 63; i >= 0; --i) {
33         x ^= (1LL << i) & k;
34         x ^= 1LL << i;
35         bool bit = (k & (1LL << i)) ==
36             0;
37         if (curr->a[bit] == nullptr ||
38             curr->a[bit]->n == 0)
39             x ^= 1LL << i, bit ^= 1;
40         curr = curr->a[bit];
41     }
42     return x;
43 }
44 };

```

4.8 segment tree

```

1 int sz = 1;
2 const int maxn = 2e6;
3 int t[maxn << 1];
4
5 void build(const vector<int>& a) {
6     for (int i = 0; i < n; ++i) {
7         t[i + sz] = a[i];
8     }
9     for (int i = sz - 1; i >= 1; --i) {
10        t[i] = max(t[i << 1], t[i << 1 | 1]);
11    }
12 }
13
14 void update(int i, int val) {
15     i += sz;
16     t[i] = val;
17     for (i >= 1; i >= 1; i >= 1) {
18         t[i] = max(t[i << 1], t[i << 1 | 1]);
19     }
20 }
21
22 int query(int r) {
23     if (t[1] < r) return 0;
24
25     int i = 1;
26     while (i < sz) {
27         if (t[i << 1] >= r) i = i << 1;
28         else i = i << 1 | 1;

```

```

29     }
30     return i - sz + 1;
31 }

```

4.9 BIT range update point query

```

1 // Fenwick Tree (Binary Indexed Tree) for
2 // Range Updates and Point Queries
3
4 template<typename T>
5 class BIT {
6     #define ALL(x) begin(x), end(x)
7 private:
8     vector<T> arr;
9     int n;
10    inline int lowbit(int x) { return x & (-
11        x); }
12    void addInternal(int s, T v) {
13        while (s > 0) {
14            arr[s] += v;
15            s -= lowbit(s);
16        }
17    }
18 public:
19    void init(int n_) {
20        n = n_;
21        arr.resize(n + 1);
22        fill(ALL(arr), 0);
23    }
24    void add(int l, int r, T v) {
25        // add v to interval [l, r], 1-based
26        addInternal(l, -v);
27        addInternal(r, v);
28    }
29    T query(int x) {
30        // value at index x
31        T res = 0;
32        while (x <= n) {
33            res += arr[x];
34            x += lowbit(x);
35        }
36        return res;
37    }
38    #undef ALL
39 };
40
41 int main() {
42     BIT<int> bit;
43     bit.init(5);
44
45     // add +3 to indices 1..3
46     bit.add(0, 3, 3);
47
48     // add +2 to indices 3..5
49     bit.add(2, 5, 2);
50
51     cout << "Value at 1 = " << bit.query(1)
52         << "\n"; // expect 3
53     cout << "Value at 3 = " << bit.query(3)
54         << "\n"; // expect 3+2=5
55     cout << "Value at 5 = " << bit.query(5)
56         << "\n"; // expect 2
57 }

```

4.10 DSU

```

1 // fast disjoint set union
2 class DSU {
3     vector<int> pa, sz;
4 public:
5     explicit DSU(int n) : pa(n, -1), sz(n,
6         1) {}
7     int find(int x) { // collapsing find
8         return pa[x] == -1 ? x : pa[x] =
9             find(pa[x]);
10    }
11    void unite(int x, int y) { // weighted
12        union
13        auto rx{find(x)}, ry{find(y)};
14        if (rx == ry) return;
15        if (sz[rx] < sz[ry]) swap(rx, ry);
16        pa[ry] = rx, sz[rx] += sz[ry];
17    }
18 };

```

5 Geometry

5.1 cross. product

```

1 // If cross(a,b,c) > 0, c is to the left of
2 // line ab
3 // If cross(a,b,c) < 0, c is to the right of
4 // line ab
5 // If cross(a,b,c) = 0, a, b, c are
6 // collinear
7
8 // point struct version
9 // 2D Point structure
10 struct Point {
11     double x, y;
12 };
13
14 // Cross product (scalar in 2D)
15 double cross(const Point& a, const Point& b,
16     const Point& c) {
17     // Computes (b - a) x (c - a)
18     return (b.x - a.x) * (c.y - a.y) -
19         (b.y - a.y) * (c.x - a.x);
20 }
21
22 // std::complex version
23 typedef std::complex<double> point;
24 #define x real()
25 #define y imag()
26
27 double cross(const point &a, const point &b)
28 {
29     return (std::conj(a) * b).imag();
30 }

```

5.2 Check Point in Polygon

```

1  /*
2  We can cast a ray from the point P going in
  any fixed direction (people usually go
  to the right).
3  If the point is located on the outside of
  the polygon the ray will intersect its
  edges an even number of times.
4  If the point is on the inside of the polygon
  then it will intersect the edge an odd
  number of times.
5  */
6
7  struct Point {
8      int x, y;
9  };
10
11  int cross(const Point& a, const Point& b,
12           const Point& c) {
13      // return (c - a) x (c - b)
14      int ans = (c.x - a.x) * (c.y - b.y) - (c
15      .x - b.x) * (c.y - a.y);
16      if (ans == 0) return 0;
17      return ans > 0 ? 1 : -1;
18  }
19
20  bool intersect(const Point& a, const Point&
21  b, const Point& c, const Point& d) {
22      int c1 = cross(a, b, c);
23      int c2 = cross(a, b, d);
24      int c3 = cross(c, d, a);
25      int c4 = cross(c, d, b);
26
27      return (c1 * c2 < 0 && c3 * c4 < 0);
28  }
29
30  bool onSegment(const Point& a, const Point&
31  b, const Point& c) {
32      return (cross(a, b, c) == 0 &&
33      min(a.x, b.x) <= c.x && c.x <=
34      max(a.x, b.x) &&
35      min(a.y, b.y) <= c.y && c.y <=
36      max(a.y, b.y));
37  }
38
39  void solve() {
40      int n, m;
41      cin >> n >> m;
42      vector<Point> poly(n);
43
44      for (int i = 0; i < n; ++i) {
45          int a, b;
46          cin >> a >> b;
47          poly[i] = {a, b};
48      }
49
50      // set a point at outside point to form
51      the ray
52      Point outside = {(int)2e9 + 7, (int)2e9
53      + 13};
54      for (int i = 0; i < m; ++i) {
55          int a, b;
56          bool found = false;
57          cin >> a >> b;

```

```

58      Point p = {a, b};
59
60      int cnt = 0;
61      if (intersect(poly.back(), poly[0],
62      p, outside)) {
63          cnt++;
64      }
65      if (onSegment(poly.back(), poly[0],
66      p)) {
67          cout << "BOUNDARY\n";
68          found = true;
69      }
70      for (int j = 0; j < n - 1 && !found;
71      ++j) {
72          if (intersect(poly[j], poly[j +
73          1], p, outside)) {
74              cnt++;
75          }
76          if (onSegment(poly[j], poly[j +
77          1], p)) {
78              cout << "BOUNDARY\n";
79              found = true;
80          }
81      }
82      if (found) continue;
83      if (cnt & 1) {
84          cout << "INSIDE\n";
85      }
86      else {
87          cout << "OUTSIDE\n";
88      }
89  }

```

5.3 Point

```

1  #pragma once
2
3  template <class T> int sgn(T x) { return (x
4  > 0) - (x < 0); }
5
6  template <class T>
7  struct Point {
8      typedef Point P;
9      T x, y;
10     explicit Point(T x=0, T y=0) : x(x), y(y)
11     {}
12
13     bool operator<(P p) const { return tie(x,y)
14     < tie(p.x,p.y); }
15     bool operator==(P p) const { return tie(x,
16     y)==tie(p.x,p.y); }
17     P operator+(P p) const { return P(x+p.x, y
18     +p.y); }
19     P operator-(P p) const { return P(x-p.x, y
20     -p.y); }
21     P operator*(T d) const { return P(x*d, y*d
22     ); }
23     P operator/(T d) const { return P(x/d, y/d
24     ); }
25     T dot(P p) const { return x*p.x + y*p.y; }
26     T cross(P p) const { return x*p.y - y*p.x; }
27
28     T cross(P a, P b) const { return (a-*this)
29     .cross(b-*this); }

```

```

18  T dist2() const { return x*x + y*y; }
19  double dist() const { return sqrt((double)
20  dist2()); }
21  // angle to x-axis in interval [-pi, pi]
22  double angle() const { return atan2(y, x); }
23
24  P unit() const { return *this/dist(); } //
25  makes dist()=1
26  P perp() const { return P(-y, x); } //
27  rotates +90 degrees
28  P normal() const { return perp().unit(); }
29  // returns point rotated 'a' radians ccw
30  around the origin
31  P rotate(double a) const {
32      return P(x*cos(a)-y*sin(a),x*sin(a)+y*
33      cos(a)); }
34
35  friend ostream& operator<<(ostream& os, P
36  p) {
37      return os << "(" << p.x << ", " << p.y <<
38      ")"; }
39  };

```

5.4 Polygon Lattice Points

```

1  /*
2  Pick's Theorem:
3  For a simple polygon with integer
4  coordinates, the area A, the number of
5  interior lattice points a
6  and the number of boundary lattice points b
7  satisfy:
8  A = a + b/2 - 1
9  */
10
11  // The number of intersections of a line with
12  // lattice points is the greatest common
13  // divisor of
14  // P1.x - P2.x and P1.y - P2.y
15  int countOnSegment(const Point& a, const
16  Point& b) {
17      int dx = abs(a.x - b.x);
18      int dy = abs(a.y - b.y);
19      return __gcd(dx, dy);
20  }
21
22  void solve() {
23      int n;
24      cin >> n;
25      vector<Point> poly(n);
26
27      for (int i = 0; i < n; ++i) {
28          cin >> poly[i].x >> poly[i].y;
29      }
30
31      // b: boundary points
32      int b = countOnSegment(poly.back(), poly
33      [0]);
34      for (int i = 0; i < n - 1; ++i) {
35          b += countOnSegment(poly[i], poly[i
36          + 1]);
37      }

```

```

38  int area2 = abs(polygonArea2(poly));
39  // a: interior points
40  int a = (area2 + 2 - b) / 2;
41  cout << a << " " << b << "\n";
42  }

```

5.5 line intersect

```

1  // 2D Point structure
2  struct Point {
3      double x, y;
4  };
5
6  // Cross product (scalar in 2D)
7  double cross(const Point& a, const Point& b,
8  const Point& c) {
9      // Computes (b - a) x (c - a)
10     return (b.x - a.x) * (c.y - a.y) -
11     (b.y - a.y) * (c.x - a.x);
12 }
13
14 // Check if value is between two others (
15 // with endpoints allowed)
16 bool between(double a, double b, double x) {
17     return min(a, b) <= x && x <= max(a, b);
18 }
19
20 // Check if point c lies on segment ab
21 bool onSegment(const Point& a, const Point&
22 b, const Point& c) {
23     return cross(a, b, c) == 0 &&
24     between(a.x, b.x, c.x) &&
25     between(a.y, b.y, c.y);
26 }
27
28 // Main intersection check
29 bool segmentsIntersect(const Point& A, const
30 Point& B, const Point& C, const
31 Point& D) {
32     double c1 = cross(A, B, C);
33     double c2 = cross(A, B, D);
34     double c3 = cross(C, D, A);
35     double c4 = cross(C, D, B);
36
37     // General case
38     if (c1 * c2 < 0 && c3 * c4 < 0) return
39     true;
40
41     // Special cases: collinear +
42     // overlapping
43     if (c1 == 0 && onSegment(A, B, C))
44         return true;
45     if (c2 == 0 && onSegment(A, B, D))
46         return true;
47     if (c3 == 0 && onSegment(C, D, A))
48         return true;
49     if (c4 == 0 && onSegment(C, D, B))
50         return true;
51
52     return false;
53 }

```

```

45 int main() {
46     Point A{0, 0}, B{4, 4};
47     Point C{0, 4}, D{4, 0};
48
49     if (segmentsIntersect(A, B, C, D))
50         cout << "Segments intersect\n";
51     else
52         cout << "Segments do not intersect\n";
53
54     return 0;
55 }

```

5.6 Polygon area

```

1 // Returns twice the signed area of a
2   polygon.
3 #pragma once
4 #include "Point.h"
5
6 template<class T>
7 T polygonArea2(vector<Point<T>>& v) {
8     T a = v.back().cross(v[0]);
9     for (int i = 0; i < (int)v.size() - 1;
10         ++i) {
11         a += v[i].cross(v[i+1]);
12     }
13     return a;
14 }
15
16 int main() {
17     // Example: square with vertices (0,0),
18     // (0,1), (1,1), (1,0)
19     vector<Point<long long>> poly = {
20         {0,0}, {0,1}, {1,1}, {1,0}
21     };
22     long long area2 = polygonArea2(poly);
23     cout << "Twice signed area = " << area2
24         << "\n";
25     cout << "Absolute area = " << abs(area2)
26         << "/ 2.0 << "\n";
27
28     return 0;
29 }

```

5.7 using std::complex

```

1 #include <iostream>
2 #include <complex>
3 #include <cmath>
4
5 typedef std::complex<double> point;
6 #define x real()
7 #define y imag()
8
9 constexpr double PI = std::acos(-1.0);
10 // conj(a) = a 的共軛 · reflection of a
   // across x-axis

```

```

11 // Basic operations
12 double dot(const point &a, const point &b) {
13     return (std::conj(a) * b).real(); }
14 double cross(const point &a, const point &b) {
15     return (std::conj(a) * b).imag(); }
16
17 // Projections / reflections / geometry
18 // helpers
19 point project_onto_vector(const point &p,
20     const point &v) {
21     return v * (dot(p, v) / std::norm(v));
22 }
23
24 point project_onto_line(const point &p,
25     const point &a, const point &b) {
26     return a + (b - a) * (dot(p - a, b - a) /
27         std::norm(b - a));
28 }
29
30 point reflect_across_line(const point &p,
31     const point &a, const point &b) {
32     return a + std::conj((p - a) / (b - a)) *
33         (b - a);
34 }
35
36 point intersection(const point &a, const
37     point &b, const point &p, const point &q)
38     {
39     double c1 = cross(p - a, b - a), c2 =
40         cross(q - a, b - a);
41     return (c1 * q - c2 * p) / (c1 - c2); //
42         // undefined if parallel (divide by
43         // zero)
44 }
45
46 double angle_between(const point &a, const
47     point &b) {
48     // angle of vector b - a
49     return std::arg(b - a);
50 }
51
52 double angle_ABC(const point &a, const point
53     &b, const point &c) {
54     double r = std::remainder(std::arg(a - b)
55         - std::arg(c - b), 2.0 * PI);
56     return std::abs(r);
57 }
58
59 int main() {
60     // sample
61     point a = 2.0; // (2,0)
62     point b(3.0, 7.0); // (3,7)
63     std::cout << a << ' ' << b << '\n'; //
64         // (2,0) (3,7)
65     std::cout << a + b << '\n'; //
66         // (5,7)
67
68     // usage examples
69     point p1(3, 2), p2(2, -7);
70     std::cout << "p1 + p2 = " << p1 + p2 <<
71         '\n'; // (5,-5)
72     std::cout << "p1 - p2 = " << p1 - p2 <<
73         '\n'; // (1,9)
74     std::cout << "3.0 * p1 = " << 3.0 * p1
75         << '\n'; // (9,6)
76 }

```

```

56 std::cout << "p1 / 5.0 = " << p1 / 5.0
57     << '\n'; // (0.6,0.4)
58
59 // dot / cross via complex
60 std::cout << "dot(p1,p2) = " << dot(p1,
61     p2) << '\n';
62 std::cout << "cross(p1,p2) = " << cross(
63     p1, p2) << '\n';
64
65 // distances, angle, rotation, polar/
66 // cartesian
67 std::cout << "squared dist = " << std:::
68     norm(p1 - p2) << '\n';
69 std::cout << "euclid dist = " << std:::
70     abs(p1 - p2) << '\n';
71 std::cout << "angle p1->p2 = " <<
72     angle_between(p1, p2) << '\n';
73 std::cout << "angle ABC = " << angle_ABC
74     (point(1,0), point(0,0), point(0,1))
75     << '\n';
76
77 // project / reflect / intersect
78 // examples
79 point v(1, 1);
80 point proj = project_onto_vector(p1, v);
81 std::cout << "project p1 onto v = " <<
82     proj << '\n';
83
84 point lineA(0,0), lineB(2,0);
85 std::cout << "project p1 onto line = "
86     << project_onto_line(p1, lineA,
87         lineB) << '\n';
88 std::cout << "reflect p1 across line = "
89     << reflect_across_line(p1, lineA,
90         lineB) << '\n';
91
92 // intersection example
93 point r1a(0,0), r1b(1,1);
94 point r2a(0,1), r2b(1,0);
95 std::cout << "intersection = " <<
96     intersection(r1a, r1b, r2a, r2b) <<
97     '\n';
98
99 // polar/cartesian and rotation
100 point polar_pt = std::polar(5.0, PI/4);
101 std::cout << "polar(5,PI/4) = " <<
102     polar_pt << '\n';
103 point rotated = p1 * std::polar(1.0, PI
104     /2); // rotate p1 by 90 degrees
105 // about origin
106 std::cout << "rotate p1 by 90deg = " <<
107     rotated << '\n';
108
109 return 0;
110 }

```

5.8 point distance from a line

```

1 // calculate area using cross product and
2 // divide by base length
3 double point_line_distance(const point &p,
4     const point &a, const point &b) {
5     // area = 0.5 * |cross(b - a, p - a)|
6 }

```

```

4 // base = |b - a|
5 // height = 2 * area / base = |cross(b -
6 // a, p - a)| / |b - a|
7 std::abs(cross(b - a, p - a)) / std::abs
8     (b - a);
9 }

```

6 Graph

6.1 Prim

```

1 // Prim's algorithm
2 vector<tuple<int, int, long long>> Prim(
3     const vector<vector<pair<int, long long
4     >>& adj) {
5     const auto& n = adj.size();
6     vector<tuple<int, int, long long>> mst
7     {};
8
9     vector<bool> found(n, false);
10    using ti = tuple<long long, int, int>;
11    priority_queue<ti, vector<ti>, greater<
12        ti>> pq{};
13    found[0] = true;
14    for (auto& [v, w] : adj[0]) pq.emplace(w,
15        0, v);
16
17    for (int i = 0; i < n - 1; ++i) {
18        int mn, u, v;
19        do {
20            tie(mn, u, v) = pq.top(), pq.pop()
21            ();
22        } while (found[v]);
23        found[v] = true, mst.emplace_back(u,
24            v, mn);
25        for (auto& [x, w] : adj[v]) pq.
26            emplace(w, v, x);
27    }
28    return mst;
29 }

```

6.2 Eulerian cycle

```

1 // Eulerian cycle in an undirected graph
2
3 vector<int> euler_cycle(vector<vector<pair<
4     int, int>>& adj, int w = 0) {
5     int n{adj.size()}; m{};
6     for (int v{0}; v < n; ++v) m += adj[v].
7         size();
8     m /= 2;
9
10    vector<int> res{};
11    stack<pair<int, int>> stk{};
12    stk.emplace(w, -1);
13    vector<int> nxt(n);
14    vector<bool> used(m);
15    while (!stk.empty()) {
16
17    }
18 }

```

```

14     auto [u, i]{stk.top()};
15     while (nxt[u] < adj[u].size() && usd
16           [adj[u][nxt[u]].second]) ++nxt[u]
17           ;
18     if (nxt[u] < adj[u].size()) {
19         auto [v, j]{adj[u][nxt[u]]};
20         ++nxt[u], usd[j] = true, stk.
21           emplace(v, j);
22     } else {
23         if (i != -1) res.push_back(i);
24         stk.pop();
25     }
26 }
27 int main() {
28     int n = 4; // number of vertices
29     vector<vector<pair<int, int>>> adj(n);
30
31     // Add edges with edge IDs
32     int eid = 0;
33     auto add_edge = [&](int u, int v) {
34         adj[u].push_back({v, eid});
35         adj[v].push_back({u, eid});
36         ++eid;
37     };
38
39     add_edge(0, 1);
40     add_edge(1, 2);
41     add_edge(2, 3);
42     add_edge(3, 0);
43
44     vector<int> cycle = euler_cycle(adj, 0);
45
46     cout << "Euler cycle (edge IDs in order)
47           : ";
48     for (int id : cycle) cout << id << " ";
49     cout << "\n";
50
51     return 0;

```

6.3 maximum weight bipartite matching

```

1 // maximum weight matching in a weighted
2 // bipartite graph
3 // Hungarian Algorithm -  $O(V^3)$ 
4 class bipartiteGraph {
5     int n;
6     vector<long long> lx, ly;
7     vector<bool> vx, vy;
8     queue<int> qu; // only X's vertices
9     vector<int> py;
10    vector<long long> dy; // smallest  $(lx[x]$ 
11      +  $ly[y] - w[x][y])$ 
12    vector<int> pdy; // & which x
13    void relax(int x) {
14        for (int y{0}; y < n; ++y)
15            if (lx[x] + ly[y] - w[x][y] < dy
16                [y])

```

```

14        dy[y] = lx[x] + ly[y] - w[x]
15            [y], pdy[y] = x;
16    }
17    void augment(int y) {
18        while (y != -1) {
19            int x{py[y]}, yy{mx[x]};
20            mx[x] = y, my[y] = x;
21            y = yy;
22        }
23    }
24    bool bfs() {
25        while (!qu.empty()) {
26            int x{qu.front()}; qu.pop();
27            for (int y{0}; y < n; ++y) {
28                if (!vy[y] && lx[x] + ly[y]
29                    == w[x][y]) {
30                    vy[y] = true, py[y] = x;
31                    if (my[y] == -1) return
32                        augment(y), true;
33                    int xx{my[y]};
34                    qu.push(x), vx[xx] =
35                        true, relax(xx);
36                }
37            }
38            return false;
39        }
40    }
41    void relabel() {
42        long long d{numeric_limits<long long>
43            >::max()};
44        for (int y{0}; y < n; ++y) if (!vy[y]
45            ) d = min(d, dy[y]);
46        for (int x{0}; x < n; ++x) if (vx[x]
47            ) lx[x] -= d;
48        for (int y{0}; y < n; ++y) if (vy[y]
49            ) ly[y] += d;
50        for (int y{0}; y < n; ++y) if (!vy[y]
51            ) dy[y] -= d;
52    }
53    bool expand() { // expand one layer with
54        new equality edges
55        for (int y{0}; y < n; ++y) {
56            if (!vy[y] && dy[y] == 0) {
57                vy[y] = true, py[y] = pdy[y]
58                ];
59                if (my[y] == -1) return
60                    augment(y), true;
61                int xx{my[y]};
62                qu.push(xx), vx[xx] = true,
63                relax(xx);
64            }
65        }
66        return false;
67    }
68    void public:
69        vector<vector<long long>> w;
70        vector<int> mx, my;
71        bipartiteGraph(int _n) : n{_n}, lx(n),
72            ly(n), vx(n), vy(n), py(n), dy(n),
73            pdy(n),
74            w(n, vector<long long>(n, 0)), mx(n,
75                -1), my(n, -1) {}
76        long long Hungarian() {
77            for (int i{0}; i < n; ++i) {
78                lx[i] = ly[i] = 0;
79                for (int j{0}; j < n; ++j)

```

```

64        lx[i] = max(lx[i], w[i][j]);
65        mx[i] = my[i] = -1;
66    }
67    for (int x{0}; x < n; ++x) {
68        fill(vx.begin(), vx.end(), false
69            );
70        fill(vy.begin(), vy.end(), false
71            );
72        fill(dy.begin(), dy.end(),
73            numeric_limits<long long>::
74            max());
75        qu = {}, qu.push(x), vx[x] =
76            true, relax(x);
77        while (bfs()) {
78            relabel();
79            if (expand()) break;
80        }
81        long long weight{0};
82        for (int x{0}; x < n; ++x) weight +=
83            w[x][mx[x]];
84        return weight;
85    }
86 }
87 // usage example:
88 int main() {
89     int n = 4;
90     bipartiteGraph g(n);
91     long long mat[4][4] = {
92         {9, 2, 7, 8},
93         {6, 4, 3, 7},
94         {5, 8, 1, 8},
95         {7, 6, 9, 4}
96     };
97     for (int i = 0; i < n; ++i)
98         for (int j = 0; j < n; ++j)
99             g.w[i][j] = mat[i][j];
100
101     long long maxWeight = g.Hungarian();
102     std::cout << "Maximum weight: " <<
103         maxWeight << "\n";
104     for (int i = 0; i < n; ++i)
105         std::cout << "X " << i << " matched
106             with Y " << g.mx[i] << "\n";

```

6.4 maximum flow

```

1 // minimum cut maximum flow
2 // relabel-to-front algorithm
3 // (an implementation of generic push-relabel
4 // algorithm)
5 // handle the capacity, c, of a vertex, v.
6 // Add a new vertex, v', and an edge v-v'
7 // with capacity c.
8 template<typename T> //requires
9     signed_integral<T>

```

```

8 void handle_vertex_capacity(int u, T c, int&
9     n, vector<tuple<int, int, T>>& e) {
10     int w{c};
11     for (auto& [_u, _v, _c] : e) if (_u == u
12         ) _u = w;
13     e.emplace_back(u, w, c);
14 }
15 template<typename T> //requires
16     signed_integral<T>
17 class flowNetwork {
18     int n, s, t;
19     vector<int> h; // height label
20     vector<T> e; // excess
21     vector<vector<int>>> adj; // adjacent
22     list
23     processed
24     list<int> lst; // topological sort of
25     admissible network
26     void push(int u, int v) {
27         T f{min(e[u], r[u][v])};
28         r[u][v] -= f, r[v][u] += f;
29         e[u] -= f, e[v] += f;
30     }
31     void relabel(int u) {
32         int mn{numeric_limits<int>::max()};
33         for (auto& v : adj[u])
34             if (r[u][v]) mn = min(mn, h[v]);
35         h[u] = mn + 1;
36     }
37     void init() {
38         fill(h.begin(), h.end(), 0);
39         fill(e.begin(), e.end(), 0);
40         lst.clear();
41         for (int u{0}; u < n; ++u) if (u !=
42             s && u != t) lst.push_back(u);
43         fill(p.begin(), p.end(), 0);
44         r = c;
45     }
46     void preflow() {
47         h[s] = n;
48         for (auto& v : adj[s]) {
49             e[v] += r[s][v], e[s] -= r[s][v]
50             ];
51             r[v][s] += r[s][v], r[s][v] = 0;
52         }
53     }
54     bool discharge(int u) {
55         bool flag{false};
56         while (e[u]) {
57             for (; p[u] < adj[u].size(); ++p
58                 [u]) {
59                 int v{adj[u][p[u]]};
60                 if (r[u][v] && h[u] == h[v]
61                     + 1) push(u, v);
62                 if (!e[u]) break;
63             }
64             if (e[u]) relabel(u), p[u] = 0,
65                 flag = true;
66         }
67         return flag; // return if relabel or
68             not
69     }
70 public:

```

6.5 maximum cardinality bipartite matching

```

62 flowNetwork(int _n) : n{_n}, s{0}, t{n -
    1}, h(n), e(n), adj(n), c(n, vector
    <T>(n)), p(n) {}
63 void set_st(int _s, int _t) {
64     s = _s, t = _t;
65 }
66 void add_edge(int u, int v, T _c) {
67     if (!c[u][v] && !c[v][u]) adj[u].
        push_back(v), adj[v].push_back(u
        );
68     c[u][v] += _c;
69 }
70 T max_flow() {
71     assert(s != t);
72
73     init();
74     preflow();
75
76     auto it{lst.begin()};
77     while (it != lst.end()) {
78         if (discharge(*it)) { // relabel
79             -to-front
80             lst.push_front(*it), lst.
                erase(it);
81             it = lst.begin();
82         }
83         it = next(it);
84     }
85     return e[t];
86 };
87
88
89 int main() {
90     using T = int;
91
92     // Build an example graph with a vertex
93     capacity on node 2.
94     int n = 4; // nodes: 0 (source), 1, 2, 3
95     (sink)
96     vector<tuple<int, int, T>> edges = {
97         {0, 1, 3},
98         {0, 2, 2},
99         {1, 2, 5},
100         {1, 3, 2},
101         {2, 3, 3}
102     };
103
104     // Enforce vertex capacity of 2 on node
105     2.
106     handle_vertex_capacity<T>(2, 2, n, edges
107     );
108
109     // Build the network and compute max
110     flow.
111     flowNetwork<T> net(n);
112     net.set_st(0, 3);
113     for (auto [u, v, cap] : edges) net.
        add_edge(u, v, cap);
114
115     cout << "Max flow = " << net.max_flow()
116     << '\n';
117 }

```

6.6 Floyd-Warshall

1 // Floyd-Warshall algorithm

```

1 class BipartiteMatching {
2     vector<bool> vis;
3     vector<int> mx{}, my{};
4     bool dfs(const vector<vector<int>>& adj,
5         int x) {
6         for (int y : adj[x]) {
7             if (vis[y]) continue;
8             vis[y] = true;
9             if (my[y] == -1 || dfs(adj, my[y]
10                )) {
11                 mx[x] = y; my[y] = x;
12                 return true;
13             }
14         }
15         return false;
16     }
17 public:
18     pair<vector<int>, vector<int>> operator
19     ()(const vector<vector<int>>& adj,
20         int ny) {
21         vis.assign(ny, false);
22         mx.assign((int)adj.size(), -1);
23         my.assign(ny, -1);
24         for (int x = 0; x < (int)adj.size();
25             ++x) {
26             fill(vis.begin(), vis.end(),
27                 false);
28             dfs(adj, x);
29         }
30         return {mx, my};
31     }
32 };
33
34 // Usage example:
35 int main() {
36     // Left partition has 4 nodes (0..3),
37     // right partition has 4 nodes (0..3).
38     vector<vector<int>> adj(4);
39     adj[0] = {0, 1};
40     adj[1] = {1, 2};
41     adj[2] = {2};
42     adj[3] = {2, 3};
43
44     BipartiteMatching bm;
45     auto res = bm(adj, 4);
46     const auto& mx = res.first;
47
48     cout << "Matched pairs (Left -> right):\n";
49     for (int i = 0; i < (int)mx.size(); ++i)
50         if (mx[i] != -1)
51             cout << i << " -> " << mx[i] <<
52             "\n";
53 }

```

```

1 template<typename T>
2 vector<vector<optional<T>>> Floyd_Warshall(
3     const vector<vector<optional<T>>>& adj)
4 {
5     const auto& n{adj.size()};
6     auto d{adj};
7
8     for (int i{0}; i < n; ++i) d[i][i] = 0;
9     for (int k{0}; k < n; ++k)
10         for (int i{0}; i < n; ++i)
11             for (int j{0}; j < n; ++j) {
12                 if (!d[i][k] || !d[k][j])
13                     continue; // no value
14                 if (!d[i][j] || d[i][j] > d[
15                     i][k].value() + d[k][j].
16                     value())
17                     d[i][j] = d[i][k].value
18                     () + d[k][j].value()
19                     ;
20             }
21     return d;
22 }

```

6.7 MST

```

1 // Kruskal's algorithm
2 vector<tuple<int, int, long long>> Kruskal(
3     int n, vector<tuple<long long, int, int>
4     >>& edges) {
5     vector<tuple<int, int, long long>> mst
6     {};
7     DSU dsu{n};
8
9     sort(edges.begin(), edges.end());
10    for (auto& [w, u, v] : edges)
11        if (dsu.find(u) != dsu.find(v))
12            dsu.unionn(u, v), mst.
13            emplace_back(u, v, w);
14    return mst;
15 }

```

6.8 Hopcroft-Karp

```

1 // maximum cardinality bipartite matching in
2 // unweighted bipartite graph
3 // Hopcroft-Karp algorithm - O(E*V)
4 class bipartiteGraph {
5     int nx, ny;
6     vector<int> dx, dy;
7     bool dfs(int y) {
8         for (auto& x : adj[y]) {
9             if (dx[x] + 1 != dy[y]) continue
10             ;
11             if (mx[x] == -1 || dfs(mx[x])) {
12                 my[y] = x, mx[x] = y;
13                 dy[y] = -1; // augmented -
14                 remove from BFS forest
15                 return true;
16             }
17         }
18     }
19 }

```

```

14 }
15 dy[y] = -1; // no augmenting path -
16     remove from BFS forest
17     return false;
18 }
19 bool bfs() {
20     fill(dx.begin(), dx.end(), -1);
21     fill(dy.begin(), dy.end(), -1);
22     queue<int> qu, qu2;
23     for (int x{0}; x < nx; ++x) // use
24         all unmatched x's as roots
25         if (mx[x] == -1) qu.push(x), dx[
26             x] = 0;
27
28     bool found{false};
29     while (!qu.empty() && !found) { //
30         stop at the level found
31         while (!qu.empty()) {
32             auto x{qu.front()}; qu.pop()
33             ;
34             for (auto& y : adj[x])
35                 if (dy[y] == -1 && my[y]
36                     != x) {
37                     dy[y] = dx[x] + 1;
38                     if (my[y] == -1)
39                         found = true;
40                     else qu2.push(my[y])
41                     , dx[my[y]] = dy
42                     [y] + 1;
43                 }
44             qu.swap(qu2);
45         }
46         return found;
47     }
48     vector<vector<int>> adjx, adjy;
49 public:
50     vector<int> mx, my;
51     bipartiteGraph(int _nx, int _ny) : nx{
52         _nx}, ny{_ny}, dx(nx), dy(ny)
53         , adjx(nx), adjy(ny), mx(nx, -1), my
54         (ny, -1) {}
55     void addEdge(int x, int y) {
56         adjx[x].push_back(y), adjy[y].
57         push_back(x);
58     }
59     int Hopcroft_Karp() {
60         int c{0};
61         while (bfs()) {
62             for (int y{0}; y < ny; ++y)
63                 if (my[y] == -1 && dy[y] !=
64                     -1) c += dfs(y);
65         }
66         return c;
67     }
68 };
69
70 // Usage example:
71 int main() {
72     // Left partition has 4 nodes (0..3),
73     // right partition has 4 nodes (0..3).
74     bipartiteGraph bg(4, 4);
75     bg.addEdge(0, 0);
76     bg.addEdge(0, 1);
77     bg.addEdge(1, 1);
78     bg.addEdge(1, 2);
79 }

```

```

66 bg.addEdge(2, 2);
67 bg.addEdge(3, 2);
68 bg.addEdge(3, 3);
69
70 int max_matching = bg.Hopcroft_Karp();
71 cout << "Maximum matching size: " <<
72   max_matching << "\n";
73 cout << "Matched pairs (Left -> right):\n";
74 for (int i = 0; i < (int)bg.mx.size();
75      ++i)
76   if (bg.mx[i] != -1)
77     cout << i << " -> " << bg.mx[i]
78     << "\n";

```

6.9 Dijkstra

```

1 // Dijkstra algorithm
2 template<typename T>
3 vector<optional<T>> Dijkstra(const vector<
4   vector<pair<int, T>>& adj, int s) {
5   const auto& n{adj.size()};
6   vector<optional<T>> d(n, nullopt);
7   d[s] = 0;
8
9   vector<bool> found(n, false);
10  using pq_t = pair<T, int>;
11  priority_queue<pq_t, vector<pq_t>,
12    greater<pq_t>> pq{};
13  pq.emplace(0, s);
14  while (!pq.empty()) {
15    auto [_, u]{pq.top()}; pq.pop();
16    if (found[u]) continue;
17    found[u] = true;
18    for (auto& [v, w] : adj[u])
19      if (!d[v] || d[v] > d[u].value()
20          + w) {
21        d[v] = d[u].value() + w;
22        pq.emplace(d[v].value(), v);
23      }
24  }
25
26  return d;

```

6.10 Dinic

```

1 int n, s, t, cnt, inf = 1e17;
2 vector<vector<int>> adj;
3 int level[100000], ptr[100000], rev[100000],
4   to[100000], cap[100000];
5 void add(int a, int b, int c) {
6   int id = cnt++;
7   to[id] = b;
8   to[rev[id]] = a;
9   cap[id] = c;
10  cap[rev[id]] = 0;
11
12  rev[id] = rev[id];

```

```

12 rev[rid] = id;
13 adj[a].push_back(id);
14 adj[b].push_back(rid);
15
16 bool bfs() {
17   memset(level, -1, sizeof(level));
18   queue<int> q;
19   q.push(s);
20   level[s] = 0;
21
22   while(!q.empty()) {
23     int curr = q.front(); q.pop();
24     for(int id: adj[curr]) {
25       int next = to[id];
26       if(cap[id] > 0 && level[next] ==
27         -1) {
28         level[next] = level[curr] +
29         1;
30         q.push(next);
31       }
32     }
33   }
34   return level[t] != -1;
35
36 int dfs(int curr, int flow) {
37   if(curr == t || flow == 0) return flow;
38   for(; ptr[curr] < adj[curr].size(); ptr[
39     curr]++) {
40     int id = adj[curr][ptr[curr]];
41     int next = to[id];
42
43     if(cap[id] > 0 && level[next] ==
44       level[curr] + 1) {
45       int temp = dfs(next, min(flow,
46         cap[id]));
47       if(temp == 0) continue;
48       int rid = rev[id];
49       cap[id] -= temp;
50       cap[rid] += temp;
51       return temp;
52     }
53   }
54   return 0;
55
56 int maxflow() {
57   int flow = 0;
58   while(bfs()) {
59     memset(ptr, 0, sizeof(ptr));
60     while(true) {
61       int temp = dfs(s, inf);
62       if(temp == 0) break;
63       flow += temp;
64     }
65   }
66   return flow;

```

6.11 maximum cardinality matching

```

1 // maximum matching
2 // Edmonds' Blossom algorithm

```

```

3 // O(VEa(E,V))
4 class graph {
5   int n;
6   vector<int> p, d, a, c1, c2;
7   // (alternating tree), (unvisited: -1,
8   // even: 0, odd: 1), (auxiliary for lca
9   //), (cross edge)
10  int label{0};
11  /* DSU */
12  vector<int> dsu_p, dsu_sz, dsu_b;
13  void dsu_reset() {
14    fill(dsu_p.begin(), dsu_p.end(), -1);
15    ;
16    fill(dsu_sz.begin(), dsu_sz.end(),
17      1);
18    iota(dsu_b.begin(), dsu_b.end(), 0);
19  }
20  int find(int x) { // collapsing find
21    return dsu_p[x] == -1 ? x : dsu_p[x]
22      = find(dsu_p[x]);
23  }
24  int base(int x) { return dsu_b[find(x)];
25  }
26  void contract(int x, int y) { //
27    weighted union
28    auto rx{find(x)}, ry{find(y)};
29    if (rx == ry) return;
30    auto b{dsu_b[rx]}; // treat x's base
31    as new base
32    if (dsu_sz[rx] < dsu_sz[ry]) swap(rx
33      , ry);
34    dsu_p[ry] = rx, dsu_sz[rx] += dsu_sz
35      [ry], dsu_b[rx] = b;
36  }
37  queue<int> qu{}; // only even vertices
38  void handle_one_side(int x, int y, int b
39    ) {
40    for (int v{base(x)}; v != b; v =
41      base(p[v])) {
42      c1[v] = x, c2[v] = y, contract(b
43        , v);
44      if (d[v] == 1) qu.push(v); //
45      odd vertex -> even vertex
46    }
47  }
48  int lca(int u, int v) {
49    ++label; // use next number in order
50    not to clear a
51    while (true) {
52      if (a[u] == label) return u;
53      a[u] = label;
54      if (p[u] != -1) u = base(p[u]);
55      swap(u, v);
56    }
57  }
58  void augment(int r, int y) {
59    if (r == y) return;
60    if (d[y] == 0) { // even vertex ->
61      odd vertex
62      // check d[y] == 0 instead of d[
63      p[y]] == 1, so (m[y], y) is
64      in the blossom
65      augment(m[y], m[c1[y]]);
66      augment(r, m[c2[y]]);

```

```

50 m[c1[y]] = c2[y], m[c2[y]] = c1[
51   y];
52 } else {
53   int x{p[y]};
54   augment(r, m[x]);
55   m[x] = y, m[y] = x;
56 }
57
58 bool bfs(int r) {
59   dsu_reset();
60   fill(d.begin(), d.end(), -1);
61
62   qu = {}, qu.push(r), d[r] = 0;
63   while (!qu.empty()) {
64     int x{qu.front()}; qu.pop();
65     for (auto& y : adj[x]) {
66       if (base(x) == base(y))
67         continue;
68       if (d[y] == -1) {
69         p[y] = x, d[y] = 1;
70         if (m[y] == -1) { //
71           augmenting path
72           augment(-1, y);
73           return true;
74         } else {
75           p[m[y]] = y, d[m[y]]
76             = 0;
77           qu.push(m[y]);
78         }
79       } else if (d[base(y)] == 0)
80         { // blossom
81           int b{lca(base(x), base(
82             y))};
83           handle_one_side(x, y, b);
84           ;
85           handle_one_side(y, x, b);
86           ;
87         }
88     }
89   }
90   return false;
91
92 public:
93   vector<vector<int>> adj;
94   vector<int> m;
95   explicit graph(int _n) : n{_n}, p(n, -1)
96     , d(n), a(n), c1(n), c2(n), dsu_p(n)
97     , dsu_sz(n), dsu_b(n), adj(n), m(n,
98     -1) {}
99   int Edmonds() {
100     int c{0};
101     for (int v{0}; v < n; ++v)
102       if (m[v] == -1 && bfs(v)) ++c;
103     return c;
104   }
105 };
106
107 // usage example:
108 int main() {
109   int n = 5;
110   graph g(n);
111   auto add = [&](int u, int v) {
112     g.adj[u].push_back(v);
113     g.adj[v].push_back(u);
114   };

```

```

105 // sample graph
106 add(0, 1);
107 add(0, 2);
108 add(1, 2); // triangle (0,1,2)
109 add(2, 3);
110 add(3, 4);
111
112 int match_size = g.Edmonds();
113 cout << "Maximum matching size: " <<
    match_size << "\n";
114 for (int i = 0; i < n; ++i)
115     if (g.m[i] != -1 && i < g.m[i])
116         cout << i << " - " << g.m[i] <<
            "\n";
117 }

```

6.12 topological sort

```

1 // topological sort 1
2 optional<vector<int>> top_sort(vector<vector
    <int>>& adj) {
3     vector<int> res{};
4     int n{static_cast<int>(adj.size())};
5     vector<int> cnt(n, 0); // predecessor
        count
6     for (int u = 0; u < n; ++u)
7         for (auto& v : adj[u]) ++cnt[v];
8
9     queue<int> qu{};
10    for (int u = 0; u < n; ++u) if (!cnt[u])
11        qu.push(u);
12    while (!qu.empty()) {
13        auto u = qu.front();
14        qu.pop();
15        res.push_back(u);
16
17        for (auto& v : adj[u])
18            if (--cnt[v]) qu.push(v);
19    }
20
21    if (res.size() != adj.size()) return
        nullopt;
22    return res;

```

6.13 Floyd's algorithm

```

1 // Floyd's Cycle-Finding Algorithm
2 // Given a function succ(x) that returns the
    successor of x,
3 a = succ(x);
4 b = succ(succ(x));
5 while (a != b) {
6     a = succ(a);
7     b = succ(succ(b));
8 }
9
10 // Find the first element in the cycle
11 a = x;
12 while (a != b) {

```

```

13     a = succ(a);
14     b = succ(b);
15 }
16 first = a;
17
18 // Find the length of the cycle
19 length = 1;
20 b = succ(a);
21 while (a != b) {
22     b = succ(b);
23     length = length + 1;
24 }
25
26 // Finding the starting point of the cycle
    from any node
27 int n;
28 vector<int> adj, res;
29 vector<vector<int>> radj;
30
31 void fill_radj(int u) {
32     for (int v : radj[u]) {
33         if (res[v] == -1) {
34             res[v] = res[u];
35             fill_radj(v);
36         }
37     }
38 }
39
40 void floyd(int x) {
41     int y = x;
42     do { // find a cycle using Floyd's
        algorithm
43         x = adj[x];
44         y = adj[adj[y]];
45     } while (y != x);
46     do { // set res[x] = x for all x along
        cycle
47         res[x] = x;
48         x = adj[x];
49     } while (y != x);
50     do { // set res'es for all x not along
        cycle
51         fill_radj(x);
52         x = adj[x];
53     } while (y != x);
54 }
55
56 void solve() {
57     cin >> n;
58     adj.resize(n);
59     radj.assign(n, {});
60     res.assign(n, -1);
61
62     for (auto& e : adj) {
63         cin >> e;
64         e--;
65     }
66
67     for (int i = 0; i < n; ++i) {
68         radj[adj[i]].push_back(i);
69     }
70
71     for (int i = 0; i < n; ++i) {
72         if (res[i] == -1) {
73             floyd(i);
74         }

```

```

75     }
76
77     for (int i = 0; i < n; ++i) {
78         cout << res[i] + 1 << " ";
79     }
80     cout << "\n";
81 }

```

6.14 stable matching

```

1 // Stable Marriage Problem (stable matching)
2 // Gale-Shapley algorithm
3 class bipartiteGraph {
4 public:
5     int n;
6     vector<vector<int>> px, py;
7     vector<int> mx, my;
8
9     bipartiteGraph(int _n) : n(_n), px(n,
        vector<int>(n)), py(n, vector<int>(n
        )), mx(n, -1), my(n, -1) {}
10    void Gale_Shapley() {
11        queue<int> qu{};
12        vector<priority_queue<pair<int, int
            >>> pq(n);
13        for (int x{0}; x < n; ++x) {
14            qu.push(x);
15            for (int y{0}; y < n; ++y) pq[x
                ].emplace(px[x][y], y);
16        }
17
18        while (!qu.empty()) {
19            auto x{qu.front()}; qu.pop();
20
21            int y;
22            do {
23                y = pq[x].top().second; pq[x
                    ].pop();
24            } while (my[y] != -1 && py[y][x]
                <= py[y][my[y]]);
25
26            if (my[y] != -1) mx[my[y]] = -1,
                qu.push(my[y]); // y
                prefers x to my[y]
                mx[x] = y, my[y] = x;
27        }
28    }
29 };
30
31 int main() {
32     int n = 3;
33     bipartiteGraph g(n);
34
35     // Men preferences (from most to Least
        preferred)
36     vector<vector<int>> men = {
37         {0, 1, 2},
38         {1, 2, 0},
39         {1, 0, 2}
40     };
41
42     // Women preferences (from most to Least
        preferred)

```

```

44     vector<vector<int>> women = {
45         {1, 0, 2},
46         {2, 1, 0},
47         {0, 1, 2}
48     };
49
50     // Convert orders to scores (higher is
        better)
51     for (int x = 0; x < n; ++x)
52         for (int pos = 0; pos < n; ++pos)
53             g.px[x][men[x][pos]] = n - pos;
54
55     for (int y = 0; y < n; ++y)
56         for (int pos = 0; pos < n; ++pos)
57             g.py[y][women[y][pos]] = n - pos;
58
59     g.Gale_Shapley();
60
61     // Output matches: man x is matched to
        woman g.mx[x]
62     for (int x = 0; x < n; ++x)
63         cout << x << " " << g.mx[x] << '\n';
64
65     return 0;
66 }

```

6.15 Bellman-Ford

```

1 // Bellman-Ford algorithm
2 template<typename T>
3 optional<vector<optional<T>>> Bellman_Ford(
    const vector<vector<pair<int, T>>& adj,
4     int s) {
5     const auto& n{adj.size()};
6     vector<optional<T>> d(n, nullopt);
7     d[s] = 0;
8
9     queue<int> qu{};
10    vector<bool> in(n, false), in2(n, false);
11
12    qu.push(s); in[s] = true;
13    for (int i{0}; i < n; ++i) { // at most
        n-1 edges
14        while (!qu.empty()) {
15            int u{qu.front()};
16            qu.pop(); in[u] = false;
17            for (auto& [v, w] : adj[u])
18                if (!d[v] || d[v] > d[u].
                    value() + w) { // relax
19                    d[v] = d[u].value() + w;
20                    if (!in2[v]) qu2.push(v);
21                    in2[v] = true;
22                }
23        }
24        qu.swap(qu2); in.swap(in2);
25
26        if (qu.empty()) return d;
27        return nullopt; // if negative cycle

```

7 Number Theory

7.1 Linear Sieve

```
1 // Calculate the smallest divisor of
  // integers in [2, maxn) in O(maxn)
2 vector<int> min_div[[]] {
3     constexpr int maxn = 400000 + 10;
4
5     vector<int> v(maxn), p;
6
7     for (int i = 2; i < maxn; ++i) {
8         if (!v[i]) {
9             v[i] = i;
10            p.push_back(i);
11        }
12        for (int j = 0; p[j] * i < maxn; ++j) {
13            v[p[j] * i] = p[j];
14            if (p[j] == v[i]) break;
15        }
16    }
17    return v;
18 }();
```

7.2 C(n,m)

```
1 ll Cnm(ll n, ll m) {
2     if (m > n / 2) m = n - m;
3     ll r[1];
4     for (ll i{1}, j{n}; i <= m; ++i, --j) r
5         *= j, r /= i;
6     return r;
7 }
```

7.3 Euler Totient 1 to n

```
1 void phi_1_to_n(int n) {
2     vector<int> phi(n + 1);
3     for (int i = 0; i <= n; ++i)
4         phi[i] = i;
5
6     for (int i = 2; i <= n; ++i) {
7         if (phi[i] == i) {
8             for (int j = i; j <= n; j += i)
9                 phi[j] -= phi[j] / i;
10        }
11    }
12 }
```

7.4 derangement (Principle of Inclusion-Exclusion)

```
1 // 1. Principle of Inclusion-Exclusion
2 // n! = n! * Σ (from k=0 to n) [((-1)^k) / (k!)]
3
4 mint c = 1;
5 for (int i = 1; i <= n; i++) {
6     c = (c * i) + (i % 2 == 1 ? -1 : 1);
7     cout << c.val() << ' ';
8 }
```

7.5 matrix template (with fast power)

```
1 template<class T> struct Matrix {
2     T **mat; int a, b;
3
4     Matrix() : a(0), b(0) {}
5     Matrix(int a_, int b_) : a(a_), b(b_) {
6         int i, j;
7         mat = new T*[a];
8         for (i = 0; i < a; ++i) {
9             mat[i] = new T[b];
10            for (j = 0; j < b; ++j) {
11                mat[i][j] = 0;
12            }
13        }
14    }
15    Matrix(const vector<vector<T>>& vt) {
16        int i, j;
17
18        *this = Matrix((int)vt.size(), (int)
19            vt[0].size());
20        for (i = 0; i < a; ++i) {
21            for (j = 0; j < b; ++j) {
22                mat[i][j] = vt[i][j];
23            }
24        }
25    }
26    Matrix operator*(const Matrix& m) {
27        int i, j, k;
28
29        assert(b == m.a);
30        Matrix r(a, m.b);
31        for (i = 0; i < a; ++i) {
32            for (j = 0; j < m.b; ++j) {
33                for (k = 0; k < b; ++k) {
34                    r.mat[i][j] += mul(mat[i][k], m.mat[k][j]);
35                }
36            }
37        }
38        return r;
39    }
40    Matrix& operator*=(const Matrix& m) {
41        return *this = (*this) * m;
42    }
43    friend Matrix power(Matrix m, long long
44        p) {
45        int i;
46        assert(m.a == m.b);
```

```
47        Matrix r(m.a, m.b);
48        for (i = 0; i < m.a; ++i) {
49            r.mat[i][i] = 1;
50        }
51        for (; p > 0; p >>= 1, m *= m) {
52            if (p & 1) {
53                r *= m;
54            }
55        }
56        return r;
57    }
58 };
59
60 int main() {
61     Matrix<int> mat(adj);
62     mat = power(mat, k); // mat^k
63     cout << mat.mat[i][j];
64 }
```

7.6 Sieve of Eratosthenes (with big num)

```
1 const int MX = 100000;
2 bool np[MX + 1];
3 vector<int> prime_numbers;
4
5 int init = []() {
6     np[0] = np[1] = true;
7     for (int i = 2; i <= MX; i++) {
8         if (!np[i]) {
9             prime_numbers.push_back(i);
10            for (int j = i; j <= MX / i; j
11                += i) { // 避免溢出的写法
12                np[i * j] = true;
13            }
14        }
15    }
16    return 0;
17 }();
18
19 bool is_prime(long long n) {
20     if (n <= MX) {
21         return !np[n];
22     }
23     for (long long p : prime_numbers) {
24         if (p * p > n) {
25             break;
26         }
27         if (n % p == 0) {
28             return false;
29         }
30     }
31     return true;
32 }
```

7.7 mod inv

```
1 // Modular inverse when mod is prime
```

```
2 long long mod_inverse(long long a, long long
3     mod) {
4     return power_mod(a, mod - 2, mod); //
5         Fermat's Little Theorem
6 }
```

7.8 derangement (DP)

```
1 // 2. DP
2 // !n = (n - 1) * (!n - 1) + !n - 2),
3 // with !0 = 1, !1 = 0
4
5 mint a = 1, b = 0;
6 cout << 0 << ' ';
7
8 for (int i = 2; i <= n; i++) {
9     mint c = (i - 1) * (a + b);
10    cout << c.val() << ' ';
11    a = b;
12    b = c;
13 }
```

7.9 Euler Totient

```
1 int phi(int n) {
2     int result = n;
3     for (int i = 2; i * i <= n; ++i) {
4         if (n % i == 0) {
5             while (n % i == 0)
6                 n /= i;
7             result -= result / i;
8         }
9     }
10    if (n > 1)
11        result -= result / n;
12    return result;
13 }
```

7.10 fast power

```
1 /* Iterative Function to calculate (a^b) %
2     mod in O(Log b) */
3 long long power_mod(long long a, long long b
4     , long long mod) {
5     long long res{1};
6     while (b > 0) {
7         if (b & 1) res = res * a % mod;
8         b >>= 1;
9         a = (a * a) % mod;
10    }
11    return res;
12 }
```

7.11 first and second mex

```

1 // Calculate first and second MEX
2 pair<int, int> calculate_mexes(vector<int>&
  nums) {
3   int n = nums.size();
4   vector<bool> seen(n + 2, false);
5
6   for (int num : nums) {
7     if (num >= 0 && num < seen.size()) {
8       seen[num] = true;
9     }
10  }
11
12  int first_mex = -1;
13  int second_mex = -1;
14
15  for (int i = 0; i < seen.size(); ++i) {
16    if (!seen[i]) {
17      if (first_mex == -1) {
18        first_mex = i;
19      } else {
20        second_mex = i;
21        break;
22      }
23    }
24  }
25
26  return {first_mex, second_mex};
27 }

```

7.12 Catalan Number

```

1 // Catalan Number
2 // C(n) = 1/(n+1) * (2n choose n) = (2n)! /
  ((n+1)! * n!)
3
4 ll catalan(int n) {
5   return fact[2 * n] * invf[n + 1] % mod *
    invf[n] % mod;
6 }

```

7.13 Chinese Remainder Theorem

```

1 // coprime (p^k)
2 pair<ll, ll> CRT(const vector<pair<ll, ll>&&
  congruences) {
3   ll M = 1, sol = 0;
4   for (auto& [m, a] : congruences) M *= m;
5   for (auto& [m, a] : congruences) {
6     ll x = M / m, y = MI(x, m);
7     sol = MA(sol, MM(MM(a, x, M), y, M),
      M);
8   }
9   return {M, sol};
10 }
11
12 // example usage
13 int main() {

```

```

14 // x ≡ 2 (mod 3)
15 // x ≡ 3 (mod 5)
16 // x ≡ 2 (mod 7)
17 vector<pair<ll, ll>> congruences = {{3,
  2}, {5, 3}, {7, 2}};
18 auto [M, sol] = CRT(congruences);
19 cout << "M: " << M << ", x: " << sol <<
  endl; // M: 105, x: 23
20 return 0;
21 }

```

7.14 mod inv (not prime)

```

1 /* exists when a and mod are coprime */
2 /* but mod is not prime */
3 long long MI(long long a, long long mod) {
4   return power_mod(a, euler_phi(mod) - 1,
    mod);
5 }

```

7.15 mod inv (not coprime)

```

1 /* a and mod are not coprime */
2 long long MI(long long a, long long mod) {
3   long long d, x, y;
4   extEcu(a, mod, d, x, y);
5   return d == 1 ? (x + mod) % mod : -1;
6 }

```

7.16 inclusion-exclusion principle

```

1 /*
2 inclusion-exclusion principle:  $U A = \sum |A_i| - \sum |A_i \cap A_j| + \sum |A_i \cap A_j \cap A_k| - \dots$ 
3
4 if  $f(n) = \sum g(d)$  for all  $d|n$ 
5 then  $g(n) = \sum \mu(d) * f(n/d)$  for all  $d|n$ 
6
7 where  $\mu(d)$  is the mobius function defined as :
8
9  $|U A| = \sum \mu(d) * |A_d|$ 
10 where  $A_d = \{x \in U : d|x\}$ 
11  $\mu(d) = 1$  if  $d$  is a product of even number of
    distinct primes
12  $\mu(d) = -1$  if  $d$  is a product of odd number of
    distinct primes
13  $\mu(d) = 0$  if  $d$  has a squared prime factor
14 */
15
16 /*
17 problem: In number theory we call an integer
    square-free if it is not divisible by a
    perfect square, except 1. You have to
    count them!

```

```

18 square-free number: An integer n is square-
    free if no  $p^2$  divides n.
19
20 */
21 const int VALMAX = 1e7;
22 int mobius[VALMAX];
23
24 int main() {
25   int test_num;
26   cin >> test_num;
27
28   mobius[1] = -1;
29   for (int i = 1; i < VALMAX; i++) {
30     if (mobius[i]) {
31       mobius[i] = -mobius[i];
32       for (int j = 2 * i; j < VALMAX; j += i)
33         { mobius[j] += mobius[i]; }
34     }
35   }
36
37   for (int t = 0; t < test_num; t++) {
38     long long n;
39     long long ans = 0;
40     cin >> n;
41     for (int i = 1; 1LL * i * i <= n; i++) {
42       ans += mobius[i] * n / ((long long)i *
        i);
43     }
44     cout << ans << '\n';
45   }
46
47   /*
48   problem: Given N cows (2 ≤ N ≤ 50,000),
    each listing five favorite ice cream
    flavors,
49   where two cows are compatible if they share
    at least one flavor,
50   determine the number of cow pairs that are
    not compatible.
51
52   ans =  $n * (n - 1) / 2 - \sum |A_i| + \sum |A_i \cap A_j| - \sum |A_i \cap A_j \cap A_k| + \dots$ 
53   where  $A_i$  is the set of cows that like
    flavor i
54   */
55
56   void solve() {
57     int n;
58     cin >> n;
59     vector<array<int, 5>> vc(n);
60
61     map<vector<int>, int> subsets;
62     for (int i = 0; i < n; ++i) {
63       for (int j = 0; j < 5; ++j) {
64         cin >> vc[i][j];
65       }
66
67       sort(vc[i].begin(), vc[i].end());
68       for (int mask = 1; mask < (1<<5); ++
        mask) {
69         vector<int> subset;
70         for (int b = 0; b < 5; ++b) {
71           if (mask & (1<<b)) {
72             subset.push_back(vc[i][b]);

```

```

73       }
74     }
75     subsets[subset]++;
76   }
77
78   int ans = n * (n - 1) / 2;
79   for (const auto& [subset, freq] :
    subsets) {
80     ans -= (subset.size() % 2 == 1 ? 1 :
      -1) * freq * (freq - 1) / 2;
81   }
82   cout << ans << "\n";
83 }
84 }

```

7.17 C(n,k) mod inverse

```

1 fac[0] = 1;
2 for (int i = 1; i <= n; ++i) {
3   fac[i] = fac[i - 1] * i % MOD;
4 }
5
6 inv_fac[n] = power_mod(fac[n], MOD - 2, MOD);
7
8 for (int i = n - 1; i >= 0; --i) {
9   inv_fac[i] = inv_fac[i + 1] * (i + 1) %
    MOD;
10 }
11
12 // C(n, k) = fac[n] * inv_fac[k] * inv_fac[n
    - k];

```

7.18 Linear Diophantine 丢番圖

```

1 // ax + by = c
2 // x0 = x * (c / gcd(a, b))
3 // y0 = y * (c / gcd(a, b))
4 // x = x0 + k * (b / gcd(a, b))
5 // y = y0 - k * (a / gcd(a, b))
6 // k is any integer
7
8 tll extendedEuclid(ll a, ll b) {
9   if (b == 0) return {a, 1, 0};
10  ll d, x1, y1;
11  tie(d, x1, y1) = extendedEuclid(b, a % b);
12  return {d, y1, x1 - (a / b) * y1};
13 }
14
15 pll linearDiophantine(ll a, ll b, ll c) {
16  ll d, x, y;
17  tie(d, x, y) = extendedEuclid(a, b);
18  if (c % d != 0) return {-1, -1}; // no
    solution
19  x *= c / d;
20  y *= c / d;
21  return {x, y};
22 }
23
24 // Example usage

```

```

25 int main() {
26     ll a = 15, b = 10, c = 35;
27     pll res = linearDiophantine(a, b, c);
28     if (res.first == -1 && res.second == -1)
29     {
30         cout << "No solution exists" << endl;
31     } else {
32         cout << "One solution is x = " <<
33             res.first << ", y = " <<
34             res.second << endl;
35     }
36     return 0;
37 }
38 ax + by = c

```

```

7 int __builtin_clz (unsigned int x)
8
9 // __builtin_ffs
10 // 返回輸入數二進制表示的最低非0位的位置。下
11 // 標從1開始計數；如果傳入0則返回0
12 int __builtin_ffs (unsigned int x)
13
14 // __builtin_popcount
15 // 是返回輸入數二進制表示中1的個數。
16 int __builtin_popcount (unsigned int x)
17
18 // __builtin_parity
19 // 返回輸入數二進制表示中1的個數的奇偶性。1
20 // 表示奇數個1。0表示偶數個1。
21 int __builtin_parity (unsigned int x)

```

```

12 n = 0;
13 vector<pii> op(q);
14 map<int, int> mp;
15 vector<int> to_val;
16
17 memset(t, 0, sizeof(t));
18
19 for (int i = 0; i < q; ++i) {
20     char c;
21     cin >> c >> op[i].second;
22     if (c == 'I') {
23         op[i].first = 0;
24         mp[op[i].second] = 1;
25     }
26     else if (c == 'D') {
27         op[i].first = 1;
28         mp[op[i].second] = 1;
29     }
30     else if (c == 'K') {
31         op[i].first = 2;
32     }
33     else {
34         op[i].first = 3;
35     }
36 }
37
38 for (auto& x : mp) {
39     x.second = n;
40     to_val.push_back(x.first);
41     n++;
42 }
43
44 build();
45
46 for (pii o : op) {
47     if (o.first == 0) {
48         int idx = mp[o.second];
49         update(idx, 1);
50     }
51     if (o.first == 1) {
52         int idx = mp[o.second];
53         update(idx, 0);
54     }
55     if (o.first == 2) {
56         int k = o.second;
57         int ans = -1;
58         int l = 0, r = n - 1;
59         while (l <= r) {
60             int m = l + (r - l) / 2;
61             int rk = query(0, m + 1);
62             if (rk >= k) {
63                 ans = m;
64                 r = m - 1;
65             }
66             else {
67                 l = m + 1;
68             }
69         }
70         if (ans == -1) cout << "invalid\n";
71         else cout << to_val[ans] << "\n";
72     }
73     if (o.first == 3) {
74         auto it = mp.lower_bound(o.
75             second);

```

```

75     if (it == mp.begin() && (it ==
76         mp.end() || o.second <= it->
77         first)) {
78         cout << "0\n";
79     }
80     else if (it == mp.end()) {
81         cout << query(0, n) << "\n";
82     }
83     else {
84         cout << query(0, it->second)
85             << "\n";
86     }
87 }

```

7.19 擴展歐基里德

```

1 /* solve x, y for ax + by = gcd(a, b) = g */
2 template<typename T>
3 void extEcu(T a, T b, T &g, T &x, T &y) {
4     if (b) extEcu(b, a % b, g, y, x), y -= x
5         * (a / b);
6     else g = a, x = 1, y = 0;
7 }

```

7.20 Sieve of Eratosthenes

```

1 void sieve(vector<int>& primes) {
2     vector<int> is_prime(INF + 1, 0);
3     is_prime[0] = 1;
4     is_prime[1] = 1;
5
6     int sq = sqrt(INF);
7
8     for (int i = 2; i <= sq; ++i) {
9         if (!is_prime[i]) {
10             primes.push_back(i);
11             for (int j = i * i; j <= INF; j
12                 += i) {
13                 is_prime[j] = 1;
14             }
15         }
16     }
17 }

```

7.21 builtin

```

1 // __builtin_ctz
2 // 返回輸入數二進制表示從最低位開始（右起）
3 // 的連續的0的個數；如果傳入0則行為未定義。
4 int __builtin_ctz (unsigned int x)
5
6 // __builtin_clz
7 // 返回輸入數二進制表示從最高位開始（左起）
8 // 的連續的0的個數；如果傳入0則行為未定義。

```

7.22 C(n,k) DP

```

1 long long binomial(long long n, long long k,
2     long long p) {
3     // dp[i][j] = iCj
4     vector<vector<long long>> dp(n + 1,
5         vector<long long>(k + 1, 0));
6
7     for (int i = 0; i <= n; ++i) {
8         dp[i][0] = 1;
9         if (i <= k) dp[i][i] = 1;
10    }
11
12    for (int i = 0; i <= n; ++i) {
13        for (int j = 1; j <= min(i, k); ++j) {
14            if (i != j) {
15                dp[i][j] = (dp[i - 1][j - 1]
16                    + dp[i - 1][j]) % p;
17            }
18        }
19    }
20
21    return dp[n][k];
22 }

```

8 Range Queries

8.1 subarray maximum sum queries

```

1 // CSES: Subarray Sum Queries
2
3 const int maxn = 2e5;
4 int n = 0;
5 int t[maxn << 1];
6
7 // Segment Tree (range sum)
8
9 void solve() {
10     int q;
11     cin >> q;

```

8.2 find first equal or greater

```

1 // CSE: Hotel Queries
2
3 int n, m;
4 int sz = 1;
5 const int maxn = 2e6;
6 int t[maxn << 1];
7
8 // Segment Tree (range max)
9
10 int query(int r) {
11     if (t[1] < r) return 0;
12
13     int i = 1;
14     while (i < sz) {
15         if (t[i << 1] >= r) i = i << 1;
16         else i = i << 1 | 1;
17     }
18     return i - sz + 1;
19 }
20
21 void solve() {
22     int r;
23     while (m--) {
24         cin >> r;
25         int idx = query(r);
26         cout << idx << " ";
27         if (idx) {
28             a[idx - 1] -= r;
29             update(idx - 1, a[idx - 1]);
30         }
31     }
32 }

```

8.3 find k-th and count less than

```

1 /*
2 INSERT(S,x), DELETE(S,x)
3 K-TH(S) : return the k-th smallest element
4           of S
5 COUNT(S,x): return the number of elements of
6           S smaller than x

```

```

5  */
6
7  // Segment Tree (range sum)
8
9  void solve() {
10     int q;
11     cin >> q;
12     n = 0;
13     vector<pii> op(q);
14     map<int, int> mp;
15     vector<int> to_val;
16
17     memset(t, 0, sizeof(t));
18
19     for (int i = 0; i < q; ++i) {
20         char c;
21         cin >> c >> op[i].second;
22         if (c == 'I') {
23             op[i].first = 0;
24             mp[op[i].second] = 1;
25         }
26         else if (c == 'D') {
27             op[i].first = 1;
28             mp[op[i].second] = 1;
29         }
30         else if (c == 'K') {
31             op[i].first = 2;
32         }
33         else {
34             op[i].first = 3;
35         }
36     }
37
38     for (auto& x : mp) {
39         x.second = n;
40         to_val.push_back(x.first);
41         n++;
42     }
43
44     build();
45
46     for (pii o : op) {
47         if (o.first == 0) {
48             int idx = mp[o.second];
49             update(idx, 1);
50         }
51         if (o.first == 1) {
52             int idx = mp[o.second];
53             update(idx, 0);
54         }
55         if (o.first == 2) {
56             int k = o.second;
57             int ans = -1;
58             int l = 0, r = n - 1;
59             while (l <= r) {
60                 int m = l + (r - l) / 2;
61                 int rk = query(0, m + 1);
62                 if (rk >= k) {
63                     ans = m;
64                     r = m - 1;
65                 }
66                 else {
67                     l = m + 1;
68                 }
69             }
70         }
71     }
72 }

```

```

70     if (ans == -1) cout << "invalid\n";
71     else cout << to_val[ans] << "\n";
72 }
73 if (o.first == 3) {
74     auto it = mp.lower_bound(o.second);
75     if (it == mp.begin() && (it == mp.end() || o.second <= it->first)) {
76         cout << "0\n";
77     }
78     else if (it == mp.end()) {
79         cout << query(0, n) << "\n";
80     }
81     else {
82         cout << query(0, it->second) << "\n";
83     }
84 }
85 }
86 }
87 }

```

9 String

9.1 Z

```

1 // 计算并返回 z 数组，其中 z[i] = |LCP(s[i:], s)|
2 vector<int> calc_z(const string& s) {
3     int n = s.size();
4     vector<int> z(n);
5     int box_l = 0, box_r = 0;
6     for (int i = 1; i < n; i++) {
7         if (i <= box_r) {
8             z[i] = min(z[i - box_l], box_r - i + 1);
9         }
10        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
11            box_l = i;
12            box_r = i + z[i];
13            z[i]++;
14        }
15    }
16    z[0] = n;
17    return z;
18 }

```

9.2 manacher

```

1 class Solution {
2 public:
3     int countSubstrings(string s) {
4         int l1 = s.size(), l2 = l1 * 2 + 1;
5         string ch = "#";

```

```

6         for(char c: s) {
7             ch = ch + c + "#";
8         }
9
10        int c = 0, r = 0, cnt = 0;
11        vector<int> p(l2);
12        for(int i = 0; i < l2; i++) {
13            p[i] = (i < r)? min(p[2 * c - i], r - i): 1;
14            while(i + p[i] < l2 && i - p[i] >= 0 && ch[i + p[i]] == ch[i - p[i]]) p[i]++;
15            if(i + p[i] > r) {
16                r = i + p[i];
17                c = i;
18            }
19            int l = p[i] - 1;
20            if(l % 2 == 0) cnt += 1 / 2;
21            else cnt += 1 / 2 + 1;
22        }
23        return cnt;
24    }
25 };

```

9.3 AC 自動機

```

1 template<char L='a',char R='z'>
2 class ac_automaton{
3     struct joe{
4         int next[R-L+1],fail,efl,ed,cnt_dp,vis;
5         joe():ed(0),cnt_dp(0),vis(0){
6             for(int i=0;i<R-L;++i)next[i]=0;
7         }
8     };
9     public:
10        std::vector<joe> S;
11        std::vector<int> q;
12        int qs,qe,vt;
13        ac_automaton():S(1),qs(0),qe(0),vt(0){
14            void clear(){
15                q.clear();
16                S.resize(1);
17                for(int i=0;i<R-L;++i)S[0].next[i]=0;
18                S[0].cnt_dp=S[0].vis=qs=qe=vt=0;
19            }
20            void insert(const char *s){
21                int o=0;
22                for(int i=0,id;s[i];++i){
23                    id=s[i]-L;
24                    if(!S[o].next[id]){
25                        S.push_back(joe());
26                        S[o].next[id]=S.size()-1;
27                    }
28                    o=S[o].next[id];
29                }
30                ++S[o].ed;
31            }
32            void build_fail(){
33                S[0].fail=S[0].efl=-1;
34                q.clear();
35                q.push_back(0);
36                ++qe;
37                while(qs!=qe){

```

```

38                int pa=q[qs++],id,t;
39                for(int i=0;i<R-L;++i){
40                    t=S[pa].next[i];
41                    if(!t)continue;
42                    id=S[pa].fail;
43                    while(~id&&!S[id].next[i])id=S[id].fail;
44                    S[t].fail=~id?S[id].next[i]:0;
45                    S[t].efl=S[t].fail?S[t].fail:S[t].fail;
46                    q.push_back(t);
47                    ++qe;
48                }
49            }
50        }
51        /*DP出每個前綴在字串s出現的次數並傳回所有字串被s匹配成功的次數O(N*M)*/
52        int match_0(const char *s){
53            int ans=0,id,p=0,i;
54            for(i=0;s[i];++i){
55                id=s[i]-L;
56                while(!S[p].next[id]&&p)S[p].fail;
57                if(!S[p].next[id])continue;
58                p=S[p].next[id];
59                ++S[p].cnt_dp; /*匹配成功則它所有後綴都可以被匹配(DP計算)*/
60            }
61            for(i=qe-1;i>=0;--i){
62                ans+=S[q[i]].cnt_dp*S[q[i]].ed;
63                if(!S[q[i]].fail)S[q[i]].fail=S[q[i]].fail;
64                cnt_dp+=S[q[i]].cnt_dp;
65            }
66            return ans;
67        }
68        /*多串匹配走efl邊並傳回所有字串被s匹配成功的次數O(N*M^1.5)*/
69        int match_1(const char *s)const{
70            int ans=0,id,p=0,t;
71            for(int i=0;s[i];++i){
72                id=s[i]-L;
73                while(!S[p].next[id]&&p)S[p].fail;
74                if(!S[p].next[id])continue;
75                p=S[p].next[id];
76                if(S[p].ed)ans+=S[p].ed;
77                for(t=S[p].efl;~t;t=S[t].efl){
78                    ans+=S[t].ed; /*因為都走efl邊所以保證匹配成功*/
79                }
80            }
81            return ans;
82        }
83        /*枚舉(s的子串nA)的所有相異字串各恰一次並傳回次數O(N*M^(1/3))*/
84        int match_2(const char *s){
85            int ans=0,id,p=0,t;
86            ++vt;
87            /*把戳記vt+=1，只要vt沒溢位，所有S[p].vis==vt就會變成false
88            這種利用vt的方法可以O(1)歸零vis陣列*/
89            for(int i=0;s[i];++i){
90                id=s[i]-L;
91                while(!S[p].next[id]&&p)S[p].fail;
92                if(!S[p].next[id])continue;
93                p=S[p].next[id];

```

```

93     if(S[p].ed&&S[p].vis!=vt){
94         S[p].vis=vt;
95         ans+=S[p].ed;
96     }
97     for(t=S[p].efl;~t&&S[t].vis!=vt;t=S[t]
98         .efl){
99         S[t].vis=vt;
100         ans+=S[t].ed; /*因為都走efl邊所以保證
101             匹配成功*/
102     }
103     return ans;
104 }
105 /*把AC自動機變成真的自動機*/
106 void evolution(){
107     for(qs=1;qs!=qe;){
108         int p=q[qs++];
109         for(int i=0;i<R-L;++i)
110             if(S[p].next[i]==0)S[p].next[i]=S[S[
111                 p].fail].next[i];
112     }
113 }

```

9.4 KMP

```

1 // 在文本串 text 中查找模式串 pattern · 返回
2 // 所有成功匹配的位置 (pattern[0] 在 text
3 // 中的下标)
4 vector<int> kmp(const string& text, const
5 string& pattern) {
6     int m = pattern.size();
7     vector<int> pi(m);
8     int cnt = 0;
9     for (int i = 1; i < m; i++) {
10         char b = pattern[i];
11         while (cnt && pattern[cnt] != b) {
12             cnt = pi[cnt - 1];
13         }
14         if (pattern[cnt] == b) {
15             cnt++;
16         }
17         pi[i] = cnt;
18     }
19     vector<int> pos;
20     cnt = 0;
21     for (int i = 0; i < text.size(); i++) {
22         char b = text[i];
23         while (cnt && pattern[cnt] != b) {
24             cnt = pi[cnt - 1];
25         }
26         if (pattern[cnt] == b) {
27             cnt++;
28         }
29         if (cnt == m) {
30             pos.push_back(i - m + 1);
31             cnt = pi[cnt - 1];
32         }
33     }
34     return pos;
35 }

```

9.5 suffix array lcp

```

1 #define radix_sort(x,y){\
2     for(i=0;i<A;++i)c[i]=0;\
3     for(i=0;i<n;++i)c[x[y[i]]]++;\
4     for(i=1;i<A;++i)c[i]+=c[i-1];\
5     for(i=n-1;~i;--i)sa[--c[x[y[i]]]]=y[i];\
6 }
7 #define AC(r,a,b)\
8     r[a]!=r[b]||a+k>n||r[a+k]!=r[b+k]
9 void suffix_array(const char *s,int n,int *
10 sa,int *rank,int *tmp,int *c){
11     int A='z'+1,i,k,id=0;
12     for(i=0;i<n;++i)rank[tmp[i]=i]=s[i];
13     radix_sort(rank,tmp);
14     for(k=1;id<n-1;k<=1){
15         for(id=0,i=n-k;i<n;++i)tmp[id++]=i;
16         for(i=0;i<n;++i)
17             if(sa[i]>=k)tmp[id++]=sa[i]-k;
18         radix_sort(rank,tmp);
19         swap(rank,tmp);
20         for(rank[sa[0]]=id=0,i=1;i<n;++i)
21             rank[sa[i]]=id+=AC(tmp,sa[i-1],sa[i]);
22         A=id+1;
23     }
24 }
25 //h: 高度數組 sa: 後綴數組 rank: 排名
26 void suffix_array_lcp(const char *s,int len,
27 int *h,int *sa,int *rank){
28     for(int i=0;i<len;++i)rank[sa[i]]=i;
29     for(int i=0,k=0;i<len;++i){
30         if(rank[i]==0)continue;
31         if(k--<0)
32             while(s[i+k]==s[sa[rank[i]-1]+k])++k;
33         h[rank[i]]=k;
34     }
35 }

```

9.6 hash

```

1 #define MAXN 1000000
2 #define mod 1073676287
3 /*mod 必須要是質數*/
4 typedef long long T;
5 char s[MAXN+5];
6 T h[MAXN+5]; /*hash陣列*/
7 T h_base[MAXN+5]; /*h_base[n]=(prime^n)%mod*/
8 void hash_init(int len,T prime){
9     h_base[0]=1;
10    for(int i=1;i<=len;++i){
11        h[i]=(h[i-1]*prime+s[i-1])%mod;
12        h_base[i]=(h_base[i-1]*prime)%mod;
13    }
14 }
15 T get_hash(int l,int r){/*閉區間寫法 · 設編號
16    為0 ~ len-1*/
17    return (h[r+1]-(h[l]*h_base[r-l+1])%mod+
18        mod)%mod;
19 }

```

9.7 minimal string rotation

```

1 int min_string_rotation(const string &s){
2     int n=s.size(),i=0,j=1,k=0;
3     while(i<n&&j<n&&k<n){
4         int t=s[(i+k)%n]-s[(j+k)%n];
5         ++k;
6         if(t){
7             if(t>0)i+=k;
8             else j+=k;
9             if(i==j)++j;
10            k=0;
11        }
12    }
13    return min(i,j); //最小循環表示法起始位置
14 }

```

9.8 reverseBWT

```

1 const int MAXN = 305, MAXC = 'Z';
2 int ranks[MAXN], tots[MAXN], first[MAXN];
3 void rankBWT(const string &bw){
4     memset(ranks,0,sizeof(int)*bw.size());
5     memset(tots,0,sizeof(tots));
6     for(size_t i=0;i<bw.size();++i)
7         ranks[i] = tots[int(bw[i])]+1;
8 }
9 void firstCol(){
10    memset(first,0,sizeof(first));
11    int totc = 0;
12    for(int c='A';c<='Z';++c){
13        if(!tots[c]) continue;
14        first[c] = totc;
15        totc += tots[c];
16    }
17 }
18 string reverseBwt(string bw,int begin){
19     rankBWT(bw), firstCol();
20     int i = begin; //原字串最後一個元素的位置
21     string res;
22     do{
23         char c = bw[i];
24         res = c + res;
25         i = first[int(c)] + ranks[i];
26     }while(i != begin);
27     return res;
28 }

```

10 Tree

10.1 Euler tour+RMQ

```

1 // Euler Tour Technique
2 class LCA {
3     const vector<vector<int>>& adj;
4     int n;
5     vector<int> d, first, euler{}, log2{};

```

```

6     vector<vector<int>> st{};
7     void dfs(int u, int w = -1, int dep = 0)
8     {
9         d[u] = dep;
10        first[u] = euler.size();
11        euler.push_back(u);
12        for (auto& v : adj[u]) {
13            if (v == w) continue;
14            dfs(v, u, dep + 1);
15            euler.push_back(u);
16        }
17    public:
18        LCA(const vector<vector<int>>& _adj, int
19            root) : adj{_adj}, n{adj.size()}, d
20            (n), first(n) {
21            dfs(root);
22
23            int tn{euler.size()};
24            log2.resize(tn + 1);
25            log2[1] = 0;
26            for (int i{2}; i <= tn; ++i) log2[i]
27                = log2[i / 2] + 1;
28
29            st.assign(tn, vector<int>(log2[tn] +
30                1));
31            for (int i{tn - 1}; i >= 0; --i) {
32                st[i][0] = euler[i];
33                for (int j{1}; i + (1 << j) <=
34                    tn; ++j) {
35                    auto& x{st[i][j - 1]};
36                    auto& y{st[i + (1 << (j - 1)
37                        )][j - 1]};
38                    st[i][j] = d[x] <= d[y] ? x
39                        : y;
40                }
41            }
42            int operator()(int u, int v) {
43                int l{first[u]}, r{first[v]};
44                if (l > r) swap(l, r);
45                ++r; // make the interval left
46                    closed right open
47
48                int j{log2[r - l]};
49                auto& x{st[l][j]};
50                auto& y{st[r - (1 << j)][j]};
51                return d[x] <= d[y] ? x : y;
52            }
53        };
54    };
55    int main() {
56        int n, q;
57        cin >> n >> q;
58        vector<vector<int>> adj(n);
59
60        for (int i = 1; i < n; ++i) {
61            int u;
62            cin >> u;
63            u--;
64            adj[u].pb(i);
65            adj[i].pb(u);
66        }
67        LCA lca(adj, 0);

```

```

62 while (q--) {
63     int u, v;
64     cin >> u >> v;
65     u--, v--;
66     cout << lca(u, v) + 1 << "\\n";
67 }
68 return 0;
69 }
70

```

10.2 HLD template

```

1 struct HLD {
2     int n, cur = 0;
3     vector<int> sz, top, dep, par, tin, tout
4         , seq;
5     vector<vector<int>> adj;
6     HLD(int n = 0) {
7         init(n);
8     }
9     void init(int n_) {
10         n = n_;
11         sz.assign(n, 1);
12         top.assign(n, -1);
13         dep.assign(n, 0);
14         par.assign(n, -1);
15         tin.assign(n, 0);
16         tout.assign(n, 0);
17         seq.assign(n, 0);
18         adj.resize(n, {});
19     }
20     void add_edge(int u, int v) { adj[u].
21         push_back(v), adj[v].push_back(u); }
22     void build(int root = 0) {
23         top[root] = root, dep[root] = 0, par
24         [root] = -1;
25         dfs1(root), dfs2(root);
26     }
27     void dfs1(int u) {
28         if (auto it = find(adj[u].begin(),
29             adj[u].end(), par[u]); it != adj
30             [u].end()) {
31             adj[u].erase(it);
32         }
33         for (auto &v : adj[u]) {
34             par[v] = u;
35             dep[v] = dep[u] + 1;
36             dfs1(v);
37             sz[u] += sz[v];
38             if (sz[v] > sz[adj[u][0]]) {
39                 swap(v, adj[u][0]);
40             }
41         }
42     }
43     void dfs2(int u) {
44         tin[u] = cur++;
45         seq[tin[u]] = u;
46         for (auto v : adj[u]) {
47             top[v] = v == adj[u][0] ? top[u]
48                 : v;
49             dfs2(v);
50         }
51         tout[u] = cur;
52     }
53 }
54

```

```

45 int lca(int u, int v) {
46     while (top[u] != top[v]) {
47         if (dep[top[u]] > dep[top[v]]) {
48             u = par[top[u]];
49         } else {
50             v = par[top[v]];
51         }
52     }
53     return dep[u] < dep[v] ? u : v;
54 }
55 int dist(int u, int v) { return dep[u] +
56     dep[v] - 2 * dep[lca(u, v)]; }
57 int jump(int u, int k) {
58     if (dep[u] < k) { return -1; }
59     int d = dep[u] - k;
60     while (dep[top[u]] > d) { u = par[
61         top[u]]; }
62     return seq[tin[u] - dep[u] + d];
63 }
64 // u is v's ancestor
65 bool is_ancestor(int u, int v) { return
66     tin[u] <= tin[v] && tin[v] < tout[u]
67     ]; }
68 // root's parent is itself
69 int rooted_parent(int r, int u) {
70     if (r == u) { return u; }
71     if (is_ancestor(r, u)) { return par[
72         u]; }
73     auto it = upper_bound(adj[u].begin(),
74         adj[u].end(), r, [&](int x,
75             int y) {
76         return tin[x] < tin[y];
77     }) - 1;
78     return *it;
79 }
80 // rooted at u, v's subtree size
81 int rooted_size(int r, int u) {
82     if (r == u) { return n; }
83     if (is_ancestor(u, r)) { return sz[u]
84         ]; }
85     return n - sz[rooted_parent(r, u)];
86 }
87 int rooted_lca(int r, int a, int b) {
88     return lca(a, b) ^ lca(a, r) ^ lca(b
89         , r); }
90 };
91 int main() {
92     ios::sync_with_stdio(false);
93     cin.tie(nullptr);
94     int n = 7;
95     HLD hld(n);
96     // build this tree:
97     //      0
98     //     / \
99     //    1  2
100    //   / \  \
101    //  3  4  5
102    //   /  \  \
103    //  6  6  6
104    hld.add_edge(0, 1);
105    hld.add_edge(0, 2);
106    hld.add_edge(1, 3);
107    hld.add_edge(1, 4);
108

```

```

101 hld.add_edge(2, 5);
102 hld.add_edge(5, 6);
103 hld.build(0); // root at node 0
104 cout << "LCA(3,4) = " << hld.lca(3,4) <<
105     "\\n"; // expected 1
106 cout << "LCA(3,6) = " << hld.lca(3,6) <<
107     "\\n"; // expected 0
108 cout << "Distance(3,4) = " << hld.dist
109     (3,4) << "\\n"; // expected 2
110 cout << "Distance(3,6) = " << hld.dist
111     (3,6) << "\\n"; // expected 4
112 cout << "Is 0 ancestor of 6? " << (hld.
113     is_ancestor(0,6) ? "yes" : "no") <<
114     "\\n"; // yes
115 return 0;
116 }
117

```

10.3 all longest path dfs

```

1 // all longest path (generalization of the
2 // tree diameter problem)
3 vector<tuple<int, int, int>> dp{};
4 // [mx1, x, mx2] the path of mx1 goes
5 // through x
6 int dfs1(int u, int p = -1) {
7     int mx = 0;
8     for (int v : adj[u]) {
9         if (v != p) {
10             int len = 1 + dfs1(v, u);
11             mx = max(mx, len);
12             auto& [mx1, x, mx2] = dp[u];
13             if (len >= mx1) {
14                 mx2 = mx1;
15                 mx1 = len;
16                 x = v;
17             }
18             else if (len > mx2) {
19                 mx2 = len;
20             }
21         }
22     }
23     return mx;
24 }
25 void dfs2(int u, int p = -1) {
26     if (p != -1) {
27         int tmx;
28         // calculate the longest path
29         // through parent
30         auto& [mx1, x, mx2] = dp[p];
31         if (x != u) tmx = mx1 + 1;
32         else tmx = mx2 + 1;
33     }
34     // update the path
35     auto& [mx1, x, mx2] = dp[u];
36     if (tmx >= mx1) {
37         mx2 = mx1;
38         mx1 = tmx;
39     }
40 }
41

```

```

39         x = p;
40     }
41     else if (tmx > mx2) {
42         mx2 = tmx;
43     }
44 }
45 }
46 for (int v : adj[u]) {
47     if (v != p) {
48         dfs2(v, u);
49     }
50 }
51 }
52 void all_longest_path() {
53     dp.resize(n);
54     dfs1(0), dfs2(0);
55 }
56

```

10.4 all longest path top sort

```

1 // all longest path in DAG
2 // 1. topological sort
3 vector<int> in(n, 0);
4 for (int i = 0; i < m; ++i) {
5     int a, b, w;
6     cin >> a >> b >> w;
7     adj[a].emplace_back(b, w);
8     in[b]++;
9 }
10 vector<int> topo; // sequence of top sort
11 queue<int> q;
12 for (int i = 0; i < n; ++i) {
13     if (in[i] == 0) {
14         q.push(i);
15     }
16 }
17 while (!q.empty()) {
18     int pa = q.front();
19     q.pop();
20     topo.push_back(pa);
21     for (auto& [child, w] : adj[pa]) {
22         in[child]--;
23         if (in[child] == 0) {
24             q.push(child);
25         }
26     }
27 }
28 // all longest path
29 vector<int> dist(n, INT_MIN);
30 vector<vector<int>> parents(n);
31 dist[0] = process[0];
32 for (int u : topo) {
33     for (auto& [v, w] : adj[u]) {
34         if (dist[v] < dist[u] + process[v] +
35             w) {
36             dist[v] = dist[u] + process[v] +
37                 w;
38             parents[v] = {u};
39         }
40     }
41 }
42

```

```

41     else if (dist[v] == dist[u] +
42             process[v] + w) {
43         parents[v].push_back(u);
44     }
45 }
46
47 cout << dist[n - 1];

```

10.5 Heavy-light decomposition (HLD)

```

1  const int maxn = 2e5;
2  int n, q;
3  int val[maxn];
4  int id[maxn];
5  int sz[maxn];
6  int parent[maxn];
7  int depth[maxn];
8  int tp[maxn];
9  vector<int> adj[maxn];
10 int timer = 0;
11
12 int dfs_sz(int u, int p) {
13     sz[u] = 1;
14     parent[u] = p;
15
16     // calculate size of each subtree
17     for (int v : adj[u]) {
18         if (v != p) {
19             depth[v] = depth[u] + 1;
20             sz[u] += dfs_sz(v, u);
21         }
22     }
23     return sz[u];
24 }
25
26 void dfs_hld(int u, int p, int top, SGT<int, MergeMax>& tree) {
27     id[u] = timer++;
28     tp[u] = top;
29     tree.update(id[u], val[u]);
30
31     // find the heavy child
32     int h_v = -1, h_sz = -1;
33     for (int v : adj[u]) {
34         if (v != p) {
35             if (h_sz < sz[v]) {
36                 h_sz = sz[v];
37                 h_v = v;
38             }
39         }
40     }
41     if (h_v == -1) {
42         return;
43     }
44
45     // continue the chain
46     dfs_hld(h_v, u, top, tree);
47     // start a new chain for each light
48     // child
49     for (int v : adj[u]) {

```

```

49         if (v != p && v != h_v) {
50             dfs_hld(v, u, v, tree);
51         }
52     }
53 }
54
55 int path(int x, int y, SGT<int, MergeMax>& tree) {
56     int ans = 0;
57     // move x and y to their top nodes until
58     // they are in the same chain
59     while (tp[x] != tp[y]) {
60         if (depth[tp[x]] < depth[tp[y]])
61             swap(x, y);
62         ans = max(ans, tree.query(id[tp[x]], id[x] + 1));
63         x = parent[tp[x]];
64     }
65     if (depth[x] > depth[y]) swap(x, y);
66     ans = max(ans, tree.query(id[x], id[y] + 1));
67     return ans;
68 }
69
70 void solve() {
71     cin >> n >> q;
72     SGT<int, MergeMax> tree(n, -1);
73
74     for (int i = 0; i < n; ++i) {
75         int a, b;
76         cin >> a >> b;
77         a--, b--;
78         adj[a].push_back(b);
79         adj[b].push_back(a);
80     }
81
82     dfs_sz(0, 0);
83     dfs_hld(0, 0, 0, tree);
84
85     cerr << "tp: ";
86     for (int i = 0; i < n; ++i) {
87         cerr << tp[i] << " ";
88     }
89     cerr << "\n";
90
91     while (q--) {
92         int mode;
93         cin >> mode;
94         if (mode == 1) {
95             int s, x;
96             cin >> s >> x;
97             s--;
98             val[s] = x;
99             tree.update(id[s], val[s]);
100         }
101         else {
102             int a, b;
103             cin >> a >> b;
104             cout << path(a - 1, b - 1, tree)
105                  << " ";
106         }
107     }
108 }

```

10.6 get kth ancestor

```

1 void dfs(int u, int p = -1) {
2     up[0][u] = (p == -1 ? u : p);
3     for (int v : adj[u]) {
4         if (v != p) {
5             dfs(v, u);
6         }
7     }
8 }
9
10 int get_kth_ancestor(int x, int k) {
11     for (int i = 0; i <= LOG; ++i) {
12         if (k & (1LL << i)) {
13             x = up[i][x];
14         }
15     }
16     return (x == 0 ? -1 : x);
17 }
18
19 void solve() {
20     int n, q;
21     cin >> n >> q;
22
23     adj[0].push_back(1);
24     for (int i = 2; i <= n; ++i) {
25         int u;
26         cin >> u;
27         adj[u].push_back(i);
28     }
29
30     dfs(0, -1);
31
32     for (int k = 1; k <= LOG; ++k) {
33         for (int v = 1; v <= n; ++v) {
34             up[k][v] = up[k - 1][up[k - 1][v]];
35         }
36     }
37
38     while (q--) {
39         int x, k;
40         cin >> x >> k;
41         cout << get_kth_ancestor(x, k) << "\n";
42     }
43 }

```

10.7 path maximum edge between u, v in tree

```

1 int depth[maxn];
2 int up[LOG][maxn];
3 int maxEdge[LOG][maxn];
4
5 void dfs(int u, int p = -1, int w = 0) {
6     up[0][u] = (p == -1 ? u : p);
7     maxEdge[0][u] = (p == -1 ? 0 : w);
8     for (auto &[v, rw] : adj[u]) {
9         if (v != p) {
10             depth[v] = depth[u] + 1;

```

```

11         dfs(v, u, rw);
12     }
13 }
14
15 int lca_weight(int a, int b) {
16     if (depth[a] < depth[b]) swap(a, b);
17     int diff = depth[a] - depth[b];
18     int ans = 0;
19
20     for (int k = 0; diff; ++k) {
21         if (diff & 1) {
22             ans = max(ans, maxEdge[k][a]);
23             a = up[k][a];
24         }
25         diff >>= 1;
26     }
27
28     if (a == b) {
29         return ans;
30     }
31     for (int k = LOG - 1; k >= 0; --k) {
32         if (up[k][a] != up[k][b]) {
33             ans = max(ans, maxEdge[k][a]);
34             ans = max(ans, maxEdge[k][b]);
35             a = up[k][a];
36             b = up[k][b];
37         }
38     }
39
40     ans = max(ans, maxEdge[0][a]);
41     ans = max(ans, maxEdge[0][b]);
42     return ans;
43 }
44
45 void solve() {
46     dfs(0, -1, 0);
47     for (int k = 1; k < LOG; ++k) {
48         for (int v = 0; v < n; ++v) {
49             up[k][v] = up[k - 1][up[k - 1][v]];
50             maxEdge[k][v] = max(maxEdge[k - 1][v], maxEdge[k - 1][up[k - 1][v]]);
51         }
52     }
53     int path_max = lca_weight(0, 1);
54 }

```

10.8 binary jumping

```

1 const int LOG = 32;
2 const int maxn = 2e5 + 2;
3
4 int depth[maxn];
5 int up[LOG][maxn];
6 vector<int> adj[maxn];
7
8 void dfs(int u, int p = -1) {
9     up[0][u] = (p == -1 ? u : p);
10    for (int v : adj[u]) {
11        if (v != p) {
12            depth[v] = depth[u] + 1;
13            dfs(v, u);

```

```

14     }
15 }
16 }
17
18 void solve() {
19     ios::sync_with_stdio(false);
20     cin.tie(nullptr);
21
22     int n, q;
23     cin >> n >> q;
24
25     for (int i = 2; i <= n; ++i) {
26         int u;
27         cin >> u;
28         adj[u].push_back(i);
29     }
30
31     dfs(1, -1); // root = 1
32
33     for (int k = 1; k < LOG; ++k) {
34         for (int v = 1; v <= n; ++v) {
35             up[k][v] = up[k - 1][up[k - 1][v]];
36         }
37     }
38
39     while (q--) {
40         int a, b;
41         cin >> a >> b;
42         cout << lca(a, b) << "\n";
43     }
44 }

```

10.9 tree diameter

```

1 int diam = 0;
2
3 int dfs(int u, int p = -1) {
4     int mx = 0;
5     for (int v : adj[u]) {
6         if (v != p) {
7             int len = 1 + dfs(v, u);
8             diam = max(diam, mx + len);
9             mx = max(mx, len);
10        }
11    }
12    return mx;
13 }

```

10.10 all longest path

```

1 int fir[maxn]; // Length of the longest
2               // downward path from u into its subtree.
3 int sec[maxn]; // Length of the second
4               // longest downward path from u
5 void dfs1(int u, int p) {

```

```

6     for (int v : adj[u]) {
7         if (v != p) {
8             dfs1(v, u);
9             if (fir[v] + 1 > fir[u]) {
10                sec[u] = fir[u];
11                fir[u] = fir[v] + 1;
12            }
13            else if (fir[v] + 1 > sec[u]) {
14                sec[u] = fir[v] + 1;
15            }
16        }
17    }
18
19    // to_p: the best path length that comes
20    // from the parent's side
21    void dfs2(int u, int p, int to_p) {
22        res[u] = max(to_p, fir[u]);
23
24        for (int v : adj[u]) {
25            if (v != p) {
26                if (fir[v] + 1 == fir[u]) {
27                    dfs2(v, u, max(to_p, sec[u]) + 1);
28                }
29                else {
30                    dfs2(v, u, res[u] + 1);
31                }
32            }
33        }
34    }
35
36    // usage
37    dfs1(1, 0);
38    dfs2(1, 0, 0);
39    // Now res[i] is the maximum distance from
40    // node i to any other node
41    for (int i = 1; i <= n; ++i) {
42        cout << res[i] << " ";
43    }

```

10.11 tree diameter (len,end)

```

1 array<int, 2> dfs(int u, int w = -1) {
2     array<int, 2> mx{0, u}; // {length,
3                             // farthest leaf}
4     for (auto& v : adj[u]) {
5         if (v == w) continue;
6         auto [len, leaf]{dfs(v, u)};
7         mx = max(mx, {len + 1, leaf});
8     }
9     return mx;
10 }
11
12 array<int, 3> tree_diameter(int a = 0) {
13     auto b{dfs(a)[1]}; // farthest node
14     // from 'a'
15     auto [l, c]{dfs(b)}; // farthest node
16     // from 'b'
17     return {1, b, c}; // {diameter
18                       // length, endpoint1, endpoint2}
19 }

```

10.12 minimum distance in functional graph

```

1 // Given a functional graph (each node has
2 // exactly one outgoing edge), answer
3 // queries about the minimum distance
4 // between two nodes, or determine that
5 // it's impossible to get from one to the
6 // other.
7 // There are three cases:
8 // 1. Both in Tree
9 // 2. Both in Cycle
10 // 3. One in Tree, one in Cycle
11
12 int main() {
13     int planet_num;
14     int query_num;
15     std::cin >> planet_num >> query_num;
16     vector<int> next(planet_num);
17     vector<vector<int>> before(planet_num);
18     for (int p = 0; p < planet_num; ++p) {
19         std::cin >> next[p];
20         next[p]--;
21         before[next[p]].push_back(p);
22     }
23
24     /*
25     * -2 = We haven't even got to processing
26     * this planet yet.
27     * -1 = This node is part of a tree.
28     * >= 0: the ID of the cycle the planet
29     * belongs to.
30     */
31     vector<int> cycle_id(planet_num, -2);
32     // Each map, given a planet #, returns the
33     // index of that planet in the
34     // cycle.
35     vector<std::map<int, int>> cycles;
36     for (int p = 0; p < planet_num; ++p) {
37         if (cycle_id[p] != -2) { continue; }
38         vector<int> path{p};
39         int at = p;
40         while (cycle_id[next[at]] == -2) {
41             at = next[at];
42             cycle_id[at] = -3; // Leave
43                               // breadcrumbs for this iteration
44         }
45         path.push_back(at);
46
47         std::map<int, int> cycle;
48         bool in_cycle = false;
49         for (int i : path) {
50             in_cycle = in_cycle || i == next[at];
51             if (in_cycle) { cycle[i] = cycle.size(); }
52         }
53         cycle_id[at] = in_cycle ? cycle.size() : -1;
54     }
55     cycles.push_back(cycle);
56
57     /*
58     * Precalculate the distance from each
59     * planet to its cycle with BFS.
60     */

```

```

61     * (cyc_dist[p] = 0) if p is part of a
62     * cycle.
63     */
64     vector<int> cyc_dist(planet_num);
65     for (int p = 0; p < planet_num; ++p) {
66         // Check if this planet is part of a
67         // cycle.
68         if (cycle_id[next[p]] == -1 || cycle_id[p] != -1) { continue; }
69         cyc_dist[p] = 1;
70         vector<int> stack(before[p]);
71         while (!stack.empty()) {
72             int curr = stack.back();
73             stack.pop_back();
74             cyc_dist[curr] = cyc_dist[next[curr]] + 1;
75             stack.insert(stack.end(), before[curr].begin(), before[curr].end());
76         }
77     }
78
79     // Initialize the binary jumping arrays.
80     int log2 = std::ceil(std::log2(planet_num));
81     vector<vector<int>> pow2_ends(planet_num, vector<int>(log2 + 1));
82     for (int p = 0; p < planet_num; ++p) {
83         pow2_ends[p][0] = next[p];
84     }
85     for (int i = 1; i <= log2; ++i) {
86         for (int p = 0; p < planet_num; ++p) {
87             pow2_ends[p][i] = pow2_ends[pow2_ends[p][i - 1]][i - 1];
88         }
89     }
90
91     /*
92     * Given a starting planet & dist, returns
93     * the planet we end up at
94     * if we use the teleporter dist times.
95     */
96     auto advance = [&](int pos, int dist) {
97         for (int pow = log2; pow >= 0; pow--) {
98             if ((dist & (1 << pow)) != 0) { pos = pow2_ends[pos][pow]; }
99         }
100         return pos;
101     };
102
103     for (int q = 0; q < query_num; ++q) {
104         int u, v; // going from u to v
105         std::cin >> u >> v;
106         u--; v--;
107         if (cycle_id[pow2_ends[u][log2]] != cycle_id[pow2_ends[v][log2]]) {
108             cout << -1 << "\n";
109             continue;
110         }
111         if (cycle_id[u] != -1 || cycle_id[v] != -1) {
112             if (cycle_id[u] == -1 && cycle_id[v] != -1) {
113                 cout << -1 << "\n";
114                 continue;
115             }
116             // Handle the 2nd & 3rd cases at the
117             // same time.

```

```

103 int dist = cyc_dist[u];
104 int u_cyc = advance(u, cyc_dist[u]);
    // The root of u's tree
105
106 std::map<int, int> &cyc = cycles[
    cycle_id[u_cyc]]; // u and v's
    cycle
107 int u_ind = cyc[u_cyc];
108 int v_ind = cyc[v];
109 int diff = u_ind <= v_ind ? v_ind -
    u_ind : cyc.size() - (u_ind -
    v_ind);
110 cout << dist + diff << '\n';
111 } else {
112 if (cyc_dist[v] > cyc_dist[u]) {
113 cout << -1 << '\n';
114 continue;
115 }
116 int diff = cyc_dist[u] - cyc_dist[v];
117 cout << (advance(u, diff) == v ? diff
    : -1) << '\n';
118 }
119 }
120 }

```

10.13 union length of two tree paths

```

1 /*
2 Problem: Given a tree and queries (a,b,c,d),
    find the size of the union of nodes on
    paths a↔b and c↔d.
3
4 Let P1 be the nodes on the path a↔b and P2
    the nodes on c↔d.
5 We want |P1 ∪ P2| = |P1| + |P2| - |P1 ∩ P2|.
6 |P1| = dist(a,b) + 1, |P2| = dist(c,d) + 1.
    So the only hard part is |P1 ∩ P2|.
7
8 Key facts for trees:
9
10 The intersection of two simple paths is
    itself either empty or a simple path.
11
12 Any endpoint of that intersection must be
    one of the LCAs among endpoint pairs of
    the two paths.
13
14 So for each query:
15
16 compute the 6 candidate LCAs,
17
18 keep those candidates that lie on both paths
    (test with distances),
19
20 if none → intersection size = 0; if one →
    intersection size = 1;
21 otherwise take the two kept candidates with
    greatest depth (they are the endpoints
    of the intersection)
22 and intersection size = dist(p,q) + 1.
23 */
24
25

```

```

26 const int LOG = 32;
27
28 const int maxn = 1e5;
29 int depth[maxn];
30 vector<int> adj[maxn];
31 int up[LOG + 1][maxn];
32 int n, q;
33
34 void dfs(int u, int p = -1) {
35 up[0][u] = (p == -1 ? u : p);
36 for (int v : adj[u]) {
37 if (v != p) {
38 depth[v] = depth[u] + 1;
39 dfs(v, u);
40 }
41 }
42 }
43
44 int dist(int a, int b) {
45 int c = lca(a, b);
46 return depth[a] + depth[b] - 2 * depth[c];
47 }
48
49 // check z on path u-v
50 bool on_path(int z, int u, int v) {
51 return dist(u, z) + dist(z, v) == dist(u,
    v);
52 }
53
54 void solve() {
55 cin >> n >> q;
56 for (int i = 0; i < n - 1; ++i) {
57 int a, b;
58 cin >> a >> b;
59 a--, b--;
60 adj[a].push_back(b);
61 adj[b].push_back(a);
62 }
63 memset(depth, 0, sizeof(depth));
64 dfs(0, -1);
65
66 for (int k = 1; k < LOG; ++k) {
67 for (int v = 0; v < n; ++v) {
68 up[k][v] = up[k - 1][up[k - 1][v]];
69 }
70 }
71
72 while (q--) {
73 int a, b, c, d;
74 cin >> a >> b >> c >> d;
75 a--, b--, c--, d--;
76 int len1 = dist(a, b) + 1;
77 int len2 = dist(c, d) + 1;
78
79 array<int, 6> cand = {
80 lca(a, b), lca(c, d),
81 lca(a, c), lca(a, d),
82 lca(b, c), lca(b, d)
83 };
84
85 vector<int> good;
86 for (int z : cand) {
87 if (on_path(z, a, b) && on_path(
    z, c, d)) {

```

```

88 good.push_back(z);
89 }
90 }
91
92 int inter = 0;
93 if (good.size() == 1) {
94 inter = 1;
95 }
96 else if (good.size() > 1) {
97 sort(good.begin(), good.end(),
    [&](int x, int y) {
98 return depth[x] > depth[y];
99 });
100 int p = good[0], qn = good[1];
101 inter = dist(p, qn) + 1;
102 }
103 cout << (len1 + len2 - inter) << "\n
    ";
104 }
105 }

```

10.14 Centroid decomposition

```

1 // Centroid Decomposition
2 // Problem: Given a tree with N nodes (1-
    indexed) and M queries. Each query is
    one of the following two types:
3 // 1 v: Paint node v red.
4 // 2 v: Print the minimal distance between
    node v and any red node.
5 // Initially, only node 1 is red.
6 // Constraints: 1 ≤ N, M ≤ 10^5
7 // Time complexity: O((N + M) log N)
8
9 vector<vector<int>> adj;
10 vector<int> subtree_size;
11 // min_dist[v] = the minimal distance
    between v and a red node
12 vector<int> min_dist;
13 vector<bool> is_removed;
14 vector<vector<pii>> ancestors;
15
16 int get_subtree_size(int u, int p = -1) {
17 subtree_size[u] = 1;
18 for (int v : adj[u]) {
19 if (v == p || is_removed[v])
20 continue;
21 subtree_size[u] += get_subtree_size(
    v, u);
22 }
23 return subtree_size[u];
24 }
25
26 int get_centroid(int u, int tree_size, int p
    = -1) {
27 for (int v : adj[u]) {
28 if (v == p || is_removed[v])
29 continue;
30 if (subtree_size[v] * 2 > tree_size)
    return get_centroid(v, tree_size,
    u);
31 }
32 }

```

```

31 }
32 return u;
33 }
34
35 // Calculate the distance between current `u`
    and the `centroid` it belongs.
36 void get_dist(int u, int centroid, int p =
    -1, int cur_dist = 1) {
37 for (int v : adj[u]) {
38 if (v == p || is_removed[v])
39 continue;
40 cur_dist++;
41 get_dist(v, centroid, u, cur_dist);
42 cur_dist--;
43 }
44 ancestors[u].emplace_back(centroid,
    cur_dist);
45 }
46
47 void build_centroid_decomp(int u = 0) {
48 int centroid = get_centroid(u,
    get_subtree_size(u));
49
50 // For all nodes in the subtree rooted
    at `centroid`, calculate their
    distance to the centroid
51 for (int v : adj[centroid]) {
52 if (is_removed[v]) continue;
53 get_dist(v, centroid, centroid);
54 }
55
56 is_removed[centroid] = true;
57 for (int v : adj[centroid]) {
58 if (is_removed[v]) continue;
59 build_centroid_decomp(v);
60 }
61 }
62
63 // Paint `node` red by updating all of its
    ancestors' minimal distances to a red
    node
64 void paint(int u) {
65 for (auto &[ancestor, dist] : ancestors[
    u]) {
66 min_dist[ancestor] = min(min_dist[
    ancestor], dist);
67 }
68 min_dist[u] = 0;
69 }
70
71 // Print the minimal distance between `u` to
    a red node.
72 void query(int u) {
73 int ans = min_dist[u];
74 for (auto &[ancestor, dist] : ancestors[
    u]) {
75 if (!dist) continue;
76 ans = min(ans, dist + min_dist[
    ancestor]);
77 }
78 cout << ans << "\n";
79 }
80
81 void solve() {
82 int N, M;
83 cin >> N >> M;

```

```

83 adj.assign(N, vector<int>());
84 for (int i = 0; i < N - 1; ++i) {
85     int a, b;
86     cin >> a >> b;
87     a--, b--;
88     adj[a].push_back(b);
89     adj[b].push_back(a);
90 }
91
92 subtree_size.assign(N, 0);
93 ancestors.assign(N, vector<pii>());
94 is_removed.assign(N, false);
95 build_centroid_decomp();
96
97 min_dist.assign(N, INF);
98 paint(0);
99 for (int i = 0; i < M; ++i) {
100     int t, v;
101     cin >> t >> v;
102     v--;
103     if (t == 1) {
104         paint(v);
105     }
106     else {
107         query(v);
108     }
109 }
110 }
111 }

```

11 default

11.1 debug

```

1 #ifdef DEBUG
2 #define dbg(...) {\
3     fprintf(stderr, "%s - %d : (%s) = ",
4         __PRETTY_FUNCTION__, __LINE__, #
5         __VA_ARGS__); \
6     _DO(__VA_ARGS__); \
7 }
8 template<typename I> void _DO(I&&x){cerr<<x
9 <<endl;}
10 template<typename I, typename...T> void _DO(I
    &&x, T&&...tail){cerr<<x<<" "; _DO(tail
    ...);}
11 #else
12 #define dbg(...)
13 #endif

```

11.2 template

```

1 // alias g++='g++ -std=c++14 -fsanitize=
2 undefined -Wall -Wextra -Wshadow -D
3 LOCAL'
4 #include <bits/stdc++.h>
5 using namespace std;

```

```

5 #ifdef LOCAL
6 #include "../debug.h"
7 #else
8 #pragma GCC optimize("O3,unroll-loops")
9 #pragma GCC target("avx2,bmi,bmi2,lzcnt,
10     popcnt")
11 #define debug(...) ((void)0)
12 #define orange(...) ((void)0)
13 #endif
14
15 #define int long long
16 #define SPEEDY ios_base::sync_with_stdio(
17     false); cin.tie(0); cout.tie(0);
18
19 void solve() {
20 }
21
22 signed main() {
23     SPEEDY;
24     return 0;
25 }

```

11.3 vimrc

```

1 set nu
2 set ts=4
3 set shiftwidth=4
4 set autoindent smartindent
5 set cindent
6
7 imap jk <ESC>
8 inoremap {<CR> {<CR>}<ESC>ko

```

12 other

12.1 Nim game

```

1 a1^a2^a3^...^an != 0 ? A win : B win

```

12.2 找小於 n 所有出現的 1 數量

```

1 current == 0 higher * factor
2 current == 1 higher * factor + lower + 1
3 other current (higher + 1) * factor

```

13 other language

13.1 python heap

```

1 import heapq
2
3 heap = [7,1,2,2]
4 heapq.heapify(heap)
5 print(heap) # [1, 2, 2, 7]
6 heapq.heappush(heap, 5)
7 print(heap) # [1, 2, 2, 7, 5]
8 print(heapq.heappop(heap)) # 1
9 print(heap) # [2, 2, 5, 7]

```

13.2 java

13.2.1 文件操作

```

1 import java.io.*;
2 import java.util.*;
3 import java.math.*;
4 import java.text.*;
5
6 public class Main{
7     public static void main(String args[]){
8         throws FileNotFoundException,
9         IOException
10         Scanner sc = new Scanner(new FileReader(
11             "a.in"));
12         PrintWriter pw = new PrintWriter(new
13             FileWriter("a.out"));
14         int n,m;
15         n=sc.nextInt();
16         m=sc.nextInt();
17
18         for(ci=1; ci<=c; ++ci){
19             pw.println("Case #" + ci + ": easy for
20                 output");
21         }
22
23         pw.close();
24         sc.close();
25     }
26 }

```

13.2.2 優先队列

```

1 PriorityQueue queue = new PriorityQueue( 1,
2     new Comparator(){
3         public int compare( Point a, Point b ){
4             if( a.x < b.x || a.x == b.x && a.y < b.y )
5                 return -1;
6             else if( a.x == b.x && a.y == b.y )
7                 return 0;
8             else return 1;
9         }
10     });

```

13.2.3 Map

```

1 Map map = new HashMap();
2 map.put("sa", "dd");
3 String str = map.get("sa").toString();
4
5 for(Object obj : map.keySet()){
6     Object value = map.get(obj );
7 }

```

13.2.4 sort

```

1 static class cmp implements Comparator{
2     public int compare(Object o1, Object o2){
3         BigInteger b1=(BigInteger)o1;
4         BigInteger b2=(BigInteger)o2;
5         return b1.compareTo(b2);
6     }
7 }
8 public static void main(String[] args)
9     throws IOException{
10     Scanner cin = new Scanner(System.in);
11     int n;
12     n=cin.nextInt();
13     BigInteger[] seg = new BigInteger[n];
14     for (int i=0;i<n;i++)
15         seg[i]=cin.nextBigInteger();
16     Arrays.sort(seg, new cmp());
17 }

```

13.3 python output

```

1 hello = 'Hello'
2 world = 7122
3 print(f'{hello} {world}') # Hello 7122
4
5 import math
6 print(f'PI is approximately {math.pi:.3f}.')
7 # PI is approximately 3.142.
8
9 print('AAA {} BBB "{}!"".format('Jin', 'Kela'
10     ''))
11 # AAA Jin BBB "Kela!"
12
13 hello = 'hello, world\n'
14 hellos = repr(hello)
15 print(hellos) # 'hello, world\n'
16
17 x = 32.5
18 y = 40000
19 print(repr((x, y, ('spam', 'eggs'))))
20 # "(32.5, 40000, ('spam', 'eggs'))"
21
22 x = 7
23 print(eval('3 * x')) # 21

```

13.4 python 大數因數分解

```

1 # 大數因數分解 (使用 Pollard's Rho 與 Miller
  -Rabin)
2 import sys, random
3 from math import gcd
4
5 # Miller-Rabin 檢定 (機率性質數判定)
6 def is_probable_prime(n, k=12):
7     if n < 2:
8         return False
9     # 先檢查一些小質數
10    small_primes =
11    [2,3,5,7,11,13,17,19,23,29]
12    for p in small_primes:
13        if n % p == 0:
14            return n == p
15    # 把 n-1 寫成 d * 2^s
16    d = n - 1
17    s = 0
18    while d % 2 == 0:
19        d //= 2
20        s += 1
21    # 重複 k 次隨機測試
22    for _ in range(k):
23        a = random.randrange(2, n - 1) # 隨
24        機挑一個測試基數
25        x = pow(a, d, n)
26        if x == 1 or x == n - 1:
27            continue
28        composite = True
29        for __ in range(s - 1):
30            x = pow(x, 2, n)
31            if x == n - 1:
32                composite = False
33                break
34        if composite:
35            return False
36        return True
37
38 # Pollard's Rho 演算法 (找非平凡因數)
39 def pollards_rho(n):
40     if n % 2 == 0:
41         return 2
42     if n % 3 == 0:
43         return 3
44     # 隨機多項式 (x^2 + c) mod n
45     while True:
46         c = random.randrange(1, n-1) #
47         隨機挑選常數 c
48         x = random.randrange(2, n-1) #
49         起始點
50         d = 1
51         while d == 1:
52             x = (pow(x, 2, n) + c) % n # x
53             -> f(x)
54             y = (pow(y, 2, n) + c) % n # y
55             -> f(f(y)) · 走兩步
56             y = (pow(y, 2, n) + c) % n
57             d = gcd(abs(x - y), n) #
58             計算兩者差的 gcd

```

```

53     if d == n:
54         失敗就重試
55         break
56     if d > 1 and d < n:
57         找到非平凡因數
58         return d
59
60 # 遞迴分解
61 def factor(n, out):
62     if n == 1:
63         return
64     if is_probable_prime(n):
65         out.append(n)
66     else:
67         d = pollards_rho(n)
68         while d is None or d == n: # 偶爾失
69             敗就重試
70             d = pollards_rho(n)
71         factor(d, out)
72         factor(n // d, out)
73
74 def main():
75     data = sys.stdin.read().strip().split()
76     if not data:
77         return
78     # 每個 token 當作一個數字
79     for token in data:
80         try:
81             n = int(token)
82         except:
83             continue
84         if n <= 1:
85             print(n)
86             continue
87         facs = []
88         factor(n, facs)
89         facs.sort()
90         # 輸出因數
91         print(" ".join(str(x) for x in facs))
92
93 if __name__ == "__main__":
94     random.seed() # 使用系統時間作為隨機種
95     子
96     main()

```

13.5 decimal

```

1 # 使用 decimal 模組來處理高精度小數運算
2 from decimal import *
3 setcontext(Context(prec=MAX_PREC, Emax=
4   MAX_EMAX, rounding=ROUND_FLOOR))
5 print(Decimal(input()) * Decimal(input()))
6
7 # 將小數轉成分數 · 方便做近似或理論分析 · 且可
8   以限制分母大小 ·
9 from fractions import Fraction
10 Fraction('3.14159').limit_denominator(10).
11 numerator # 22
12
13 # 設定精確度

```

```

11 from decimal import Decimal, getcontext
12
13 # 精確位數設定
14 getcontext().prec = 70
15
16 n = 100
17 # 指定 n 為高精確度的物件
18 n = Decimal(n)
19 n /= 7
20 print(n)
21
22 # 將小數轉成分數
23 from fractions import Fraction
24
25 n = 1.5654
26 # 建立一個轉換物件
27 n = Fraction(n)
28
29 print(n)

```

13.6 python 大數排序

```

1 # 大數排序
2 # Line one n : 多少數字
3 # next line : 依序輸入每行一個
4 # sort : sort + Lambda
5 from sys import stdin
6
7 data = stdin.read().splitlines()
8
9 i = 0
10
11 while(i < len(data)):
12     T = int(data[i].strip())
13     i += 1
14     temp = [int(data[i + index].strip()) for
15             index in range(T)]
16     i += T
17     temp = sorted(temp, key = lambda x : (x))
18     for ele in temp:
19         print(ele)

```

13.7 python 大數計算 2

```

1 # 單行輸入
2 # format : n1, operation, n2
3 from sys import stdin
4
5 data = stdin.read().splitlines()
6
7 limit = len(data)
8 i = 0
9
10 while(i < limit):
11     a, operation, b = map(str, data[i].split
12     ())
13     a, b = int(a), int(b)
14     i += 1

```

```

14 if(operation == '+'):
15     print(int(a + b))
16 elif(operation == '-'):
17     print(int(a - b))
18 elif(operation == '*'):
19     print(int(a * b))
20 else:
21     print(int(a // b))

```

13.8 python input

```

1 ans = sum(map(float, input().split()))
2 # input: 1.1 2.2 3.3 4.4 5.5
3 print(ans) # 16.5
4
5 (n, m) = map(int, input().split()) # 300 200
6 print(n * m) # 60000
7
8 Arr = list(map(int, input().split()))
9 # input: 1 2 3 4 5
10 print(Arr) # [1, 2, 3, 4, 5]

```

13.9 python 大數計算

```

1 # 讀取多行輸入
2 # Line one first number
3 # Line two operation
4 # Line three second number
5 from sys import stdin
6
7 data = stdin.read().splitlines()
8
9 limit = len(data)
10 i = 0
11 while(i < limit):
12     a = int(data[i].strip())
13     i += 1
14     operation = data[i].strip()
15     i += 1
16     b = int(data[i].strip())
17     i += 1
18     if(operation == '*'):
19         print(int(a * b))
20     else:
21         print(int(a / b))

```

13.10 base-6 pow

```

1 import sys
2
3 sys.setrecursionlimit(2000000)
4
5 def solve():
6     S = sys.stdin.readline().strip()
7     try:
8         N = int(S)

```

```
9 | except ValueError:
10 |     N = 0
11 | if N == 0:
12 |     print(1)
13 |     return
14 | low = 1
15 | high = 700000 # Safely larger than
16 |             500000 * 1.3
17 | ans = high + 1
18 | while low <= high:
19 |     L_mid = (low + high) // 2
20 |     if L_mid <= 0: # Avoid 6^0 or 6^
21 |         negative
22 |         low = L_mid + 1
23 |         continue
24 |     val = pow(6, L_mid)
25 |     if val > N:
26 |         ans = L_mid
27 |         high = L_mid - 1
28 |     else:
29 |         low = L_mid + 1
30 | print(ans)
31 | solve()
```

14 zformula

14.1 formula

14.1.1 Pick 公式

給定頂點坐標均是整點的簡單多邊形，面積 = 內部格點數 + 邊上格點數/2-1

14.1.2 圖論

- 1. 對於平面圖， $F = E - V + C + 1$ ，C 是連通分量數
- 2. 對於平面圖， $E \leq 3V - 6$
- 3. 對於連通圖 G，最大獨立點集的大小設為 $I(G)$ ，最大匹配大小設為 $M(G)$ ，最小點覆蓋設為 $Cv(G)$ ，最小邊覆蓋設為 $Ce(G)$ 。對於任意連通圖：

- (a) $I(G) + Cv(G) = |V|$
- (b) $M(G) + Ce(G) = |V|$

- 4. 對於連通二分圖：

- (a) $I(G) = Cv(G)$
- (b) $M(G) = Ce(G)$

14.1.3 學長公式

- 1. $\sum_{d|n} \phi(n) = n$
- 2. Harmonic series $H_n = \ln(n) + \gamma + 1/(2n) - 1/(12n^2) + 1/(120n^4)$
- 3. $\gamma = 0.57721566490153286060651209008240243104215$
- 4. 選轉矩陣 $M(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$

14.1.4 基本數論

- 1. $\sum_{d|n} \mu(n) = [n == 1]$
- 2. $g(m) = \sum_{d|m} f(d) \Leftrightarrow f(m) = \sum_{d|m} \mu(d) \times g(m/d)$
- 3. $\sum_{i=1}^n \sum_{j=1}^m \text{互質數量} = \sum \mu(d) \lfloor \frac{n}{d} \rfloor \lfloor \frac{m}{d} \rfloor$
- 4. $\sum_{i=1}^n \sum_{j=1}^m lcm(i, j) = n \sum_{d|n} d \times \phi(d)$

14.1.5 排組公式

- 1. k 卡特蘭 $\frac{C_n^n}{n(k-1)+1} \cdot C_m^n = \frac{n!}{m!(n-m)!}$
- 2. $H(n, m) \cong x_1 + x_2 \dots + x_n = k, num = C_k^{n+k-1}$
- 3. Stirling number of 2^{nd} , n 入分 k 組方法數目
 - (a) $S(0, 0) = S(n, n) = 1$
 - (b) $S(n, 0) = 0$
 - (c) $S(n, k) = kS(n - 1, k) + S(n - 1, k - 1)$

- 4. Bell number, n 入分任意多組方法數目

- (a) $B_0 = 1$
- (b) $B_n = \sum_{i=0}^n S(n, i)$
- (c) $B_{n+1} = \sum_{k=0}^n C_k^n B_k$
- (d) $B_{p+n} \equiv B_n + B_{n+1} \text{ mod } p$, p is prime
- (e) $B_p^{m+n} \equiv mB_n + B_{n+1} \text{ mod } p$, p is prime
- (f) From $B_0 : 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975$

- 5. Derangement, 錯排, 沒有人在自己位置上

- (a) $D_n = n!(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} \dots + (-1)^n \frac{1}{n!})$
- (b) $D_n = (n - 1)(D_{n-1} + D_{n-2}), D_0 = 1, D_1 = 0$
- (c) From $D_0 : 1, 0, 1, 2, 9, 44, 265, 1854, 14833, 133496$

- 6. Binomial Equality

- (a) $\sum_k \binom{r}{m+k} \binom{s}{n-k} = \binom{r+s}{m+n}$
- (b) $\sum_k \binom{l}{m+k} \binom{s}{n+k} = \binom{l+s}{l-m+n}$
- (c) $\sum_k \binom{l}{m+k} \binom{s+k}{n} (-1)^k = (-1)^{l+m} \binom{s-m}{n-l}$
- (d) $\sum_{k \leq l} \binom{l-k}{m} \binom{s}{k-n} (-1)^k = (-1)^{l+m} \binom{s-m-1}{l-n-m}$
- (e) $\sum_{0 \leq k \leq l} \binom{l-k}{m} \binom{q+k}{n} = \binom{l+q+1}{m+n+1}$
- (f) $\binom{r}{k} = (-1)^k \binom{k-r-1}{k}$
- (g) $\binom{r}{m} \binom{m}{k} = \binom{r}{k} \binom{r-k}{m-k}$
- (h) $\sum_{k \leq n} \binom{r+k}{k} = \binom{r+n+1}{n}$
- (i) $\sum_{0 \leq k \leq n} \binom{k}{m} = \binom{n+1}{m+1}$
- (j) $\sum_{k \leq m} \binom{m+r}{k} x^k y^{m-k} = \sum_{k \leq m} \binom{-r}{k} (-x)^k (x+y)^{m-k}$

14.1.6 冪次, 冪次和

- 1. $a^{b\%p} P = a^{b\% \varphi(p) + \varphi(p)}, b \geq \varphi(p)$
- 2. $1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^4}{4} + \frac{n^3}{2} + \frac{n^2}{4}$
- 3. $1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n^5}{5} + \frac{n^4}{2} + \frac{n^3}{3} - \frac{n}{30}$
- 4. $1^5 + 2^5 + 3^5 + \dots + n^5 = \frac{n^6}{6} + \frac{n^5}{2} + \frac{5n^4}{12} - \frac{n^2}{12}$
- 5. $0^k + 1^k + 2^k + \dots + n^k = P(k), P(k) = \frac{(n+1)^{k+1} - \sum_{i=0}^{k-1} C_i^{k+1} P(i)}{k+1}, P(0) = n + 1$
- 6. $\sum_{k=0}^{m-1} k^n = \frac{1}{n+1} \sum_{k=0}^n C_k^{n+1} B_k m^{n+1-k}$
- 7. $\sum_{j=0}^m C_j^{m+1} B_j = 0, B_0 = 1$
- 8. 除了 $B_1 = -1/2$ ，剩下的奇數項都是 0
- 9. $B_2 = 1/6, B_4 = -1/30, B_6 = 1/42, B_8 = -1/30, B_{10} = 5/66, B_{12} = -691/2730, B_{14} = 7/6, B_{16} = -3617/510, B_{18} = 43867/798, B_{20} = -174611/330,$

14.1.7 循環小數轉分數

- 1. 若 $x = 0.\overline{a}$ ，則

$$x = \frac{a}{\underbrace{99 \dots 9}_{k \text{ digits}}}$$

其中 a 為循環節，k 為循環節的位數。

- 2. 例子：

$$0.\overline{37} = \frac{37}{99}$$

$$0.\overline{5} = \frac{5}{9}$$

- 3. 純循環小數：若 $x = 0.\overline{a}$ ，其中 a 為循環節、長度為 k，

$$x = \frac{a}{\underbrace{99 \dots 9}_{k \text{ digits}}}$$

例：

$$0.\overline{37} = \frac{37}{99}, \quad 0.\overline{5} = \frac{5}{9}$$

- 4. 混循環小數：若 $x = 0.b\overline{a}$ ，其中 b 為前綴、長度 m，a 為循環節、長度 k，

$$x = \frac{(b \cdot 10^k + a) - b}{10^m(10^k - 1)}$$

例：

$$0.12\overline{3} = \frac{(12 \cdot 10^1 + 3) - 12}{10^2(10^1 - 1)} = \frac{123 - 12}{100 \cdot 9} = \frac{111}{900} = \frac{37}{300}$$

14.1.8 常見級數與組合公式

- 1. 平方和公式：

$$1^2 + 2^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

$$4^2 + 6^2 + \dots + (2n)^2 = \frac{(2n)(n+1)(2n+1)}{3}$$

$$1^2 + 3^2 + \dots + (2n+1)^2 = \frac{n(2n-1)(2n+1)}{3}$$

- 2. 立方和公式：

$$1^3 + 2^3 + \dots + n^3 = \frac{n^4 + 2n^3 + n^2}{4}$$

- 3. 四次方和公式：

$$1^4 + 2^4 + \dots + n^4 = \frac{6n^5 + 15n^4 + 10n^3 - n}{30}$$

- 4. 五次方和公式：

$$1^5 + 2^5 + \dots + n^5 = \frac{2n^6 + 6n^5 + 5n^4 - n^2}{12}$$

- 5. 六次方和公式：

$$1^6 + 2^6 + \dots + n^6 = \frac{6n^7 + 21n^6 + 21n^5 - 7n^3 + n}{42}$$

- 6. 七次方和公式：

$$1^7 + 2^7 + \dots + n^7 = \frac{3n^8 + 12n^7 + 14n^6 - 7n^4 + 2n^2}{24}$$

- 7. 八次方和公式：

$$\begin{aligned} &1^8 + 2^8 + \dots + n^8 \\ &= \frac{10n^9 + 45n^8 + 60n^7 - 42n^5 + 20n^3 - 3n}{90} \end{aligned}$$

- 8. 九次方和公式：

$$\begin{aligned} &1^9 + 2^9 + \dots + n^9 \\ &= \frac{2n^{10} + 10n^9 + 15n^8 - 14n^6 + 10n^4 - 3n^2}{20} \end{aligned}$$

- 9. 十次方和公式：

$$\begin{aligned} &1^{10} + 2^{10} + \dots + n^{10} \\ &= \frac{6n^{11} + 33n^{10} + 55n^9 - 66n^7 + 66n^5 - 33n^3 + 5n}{66} \end{aligned}$$

- 10. 等比級數：

$$S = a \cdot \frac{r^n - 1}{r - 1}$$

ACM ICPC Team Reference - BogoSort

Contents

1 Algorithm	1	4.7 Trie (Prefix tree)	5	7.8 derangement (DP)	12	10.12 minimum distance in functional graph	19
1.1 LIS	1	4.8 segment tree	5	7.9 Euler Totient	12	10.13 union length of two tree paths	20
1.2 LCS	1	4.9 BIT range update point query	5	7.10 fast power	12	10.14 Centroid decomposition	20
2 Basic	1	4.10 DSU	5	7.11 first and second mex	13	11 default	21
2.1 mod helper function	1	5 Geometry	5	7.12 Catalan Number	13	11.1 debug	21
2.2 self-defined-pq-operator	1	5.1 cross. product	5	7.13 Chinese Remainder Theorem	13	11.2 template	21
2.3 generating all subsets	1	5.2 Check Point in Polygon	6	7.14 mod inv (not prime)	13	11.3 vimrc	21
2.4 tuple	1	5.3 Point	6	7.15 mod inv (not coprime)	13	12 other	21
2.5 memset	1	5.4 Polygon Lattice Points	6	7.16 inclusion-exclusion principle	13	12.1 Nim game	21
2.6 submask enumeration	1	5.5 line intersect	6	7.17 C(n,k) mod inverse	13	12.2 找小於 n 所有出現的 1 數量	21
2.7 stringstream split by comma	1	5.6 Polygon area	7	7.18 Linear Diophantine 丟番圖	13	13 other language	21
3 DP	1	5.7 using std::complex	7	7.19 擴展歐基里德	14	13.1 python heap	21
3.1 walk	1	5.8 point distance from a line	7	7.20 Sieve of Eratosthenes	14	13.2 java	21
3.2 grouping	1	6 Graph	7	7.21 builtin	14	13.2.1 文件操作	21
3.3 matching	2	6.1 Prim	7	7.22 C(n,k) DP	14	13.2.2 優先隊列	21
3.4 elevator rides	2	6.2 Eulerian cycle	7	8 Range Queries	14	13.2.3 Map	21
3.5 digit sum	2	6.3 maximum weight bipartite matching	8	8.1 subarray maximum sum queries	14	13.2.4 sort	21
3.6 sushi	2	6.4 maximum flow	8	8.2 find first equal or greater	14	13.3 python output	21
3.7 permutation	2	6.5 maximum cardinality bipartite matching	9	8.3 find k-th and count less than	14	13.4 python 大數因數分解	22
3.8 independent set	3	6.6 Floyd-Warshall	9	9 String	15	13.5 decimal	22
3.9 counting tower	3	6.7 MST	9	9.1 Z	15	13.6 python 大數排序	22
3.10 counting numbers	3	6.8 Hopcroft-Karp	9	9.2 manacher	15	13.7 python 大數計算 2	22
4 Data Structure	3	6.9 Dijkstra	10	9.3 AC 自動機	15	13.8 python input	22
4.1 undo disjoint set	3	6.10 Dinic	10	9.4 KMP	16	13.9 python 大數計算	22
4.2 lazy propagation	4	6.11 maximum cardinality matching	10	9.5 suffix array lcp	16	13.10 base-6 pow	22
4.3 Lazy heap	4	6.12 topological sort	11	9.6 hash	16	14 zformula	23
4.4 Fenwick tree (BIT)	4	6.13 Floyd's algorithm	11	9.7 minimal string rotation	16	14.1 formula	23
4.5 BIT - range update + range prefix sum	4	6.14 stable matching	11	9.8 reverseBWT	16	14.1.1 Pick 公式	23
4.6 Order Statistics Tree	5	6.15 Bellman-Ford	11	10 Tree	16	14.1.2 圖論	23
		7 Number Theory	12	10.1 Euler tour+RMQ	16	14.1.3 學長公式	23
		7.1 Linear Sieve	12	10.2 HLD template	17	14.1.4 基本數論	23
		7.2 C(n,m)	12	10.3 all longest path dfs	17	14.1.5 排組公式	23
		7.3 Euler Totient 1 to n	12	10.4 all longest path top sort	17	14.1.6 冪次, 冪次和	23
		7.4 derangement (Principle of Inclusion-Exclusion)	12	10.5 Heavy-light decomposition (HLD)	18	14.1.7 循環小數轉分數	23
		7.5 matrix template (with fast power)	12	10.6 get kth ancestor	18	14.1.8 常見級數與組合公式	23
		7.6 Sieve of Eratosthenes (with big num)	12	10.7 path maximum edge between u, v in tree	18	14.1.9 位元運算	24
		7.7 mod inv	12	10.8 binary jumping	18	14.1.10 除數與倍數	24
				10.9 tree diameter	19	14.1.11 數論公式	24
				10.10 all longest path	19	15 Интернационал	24
				10.11 tree diameter (len,end)	19	15.1 保佑	24